

Gild the Tulip: Ergodicity-Aware Portfolio Construction Under Fat Tails

Kieran Hannigan QIP Winter 2026 · Prof. Juhani Linnainmaa · Tuck School of Business

In my final project for Investments ("Buy the Tulip," Hannigan et al. 2026), I critiqued CAPM's Gaussian mean-variance framework. I showed that market returns are leptokurtic, negatively skewed, and better described by a Lévy stable distribution with $\alpha \approx 1.55$. I recommended fractional Kelly with CVaR constraints as the honest alternative to mean-variance optimization.

This notebook asks the natural follow-up: what does a tail-aware portfolio actually look like, and does the answer change when you test it outside the most successful equity market in history?

Part 1 constructs and compares five portfolios on US Fama-French data (1963-2024). The conventional result: 80/20 dominates on terminal wealth in every historical scenario. The barbell trades CAGR for predictability and drawdown control.

Part 2 tests whether this result is an artifact of US survivorship bias. Using the Jordà-Schularick-Taylor Macrohistory Database (16 countries, 1870-2020) and gold as an alternative safe asset, I find that the answer depends on which monetary era you consider — and that the choice of *safe asset* matters more than the choice of *risky asset*.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from scipy import stats
from statsmodels.stats.stattools import jarque_bera
import statsmodels.api as sm
from datetime import datetime
import requests
from io import BytesIO, StringIO
import zipfile
import warnings
from matplotlib.colors import TwoSlopeNorm
from matplotlib.ticker import PercentFormatter

try:
    plt.style.use('seaborn-v0_8-whitegrid')
except OSError:
    plt.style.use('seaborn-whitegrid')

plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['figure.dpi'] = 100
warnings.filterwarnings('ignore')

COLORS = {
    '60/40': 'steelblue',
    '80/20': 'darkorange',
    'Five-Factor': 'mediumorchid',
    'Barbell': 'seagreen',
    'Kelly Barbell': 'firebrick',
    'Barbell (gold)': 'goldenrod',
}

print('All imports successful.')
```

All imports successful.

Part 1: The US Case

Ergodicity, the Volatility Tax, and the Barbell Rationale

The standard finance framework evaluates gambles by their **ensemble average** — the expected value across many parallel investors at a single point in time. But a real investor lives through *sequential* time. The distinction matters whenever returns are multiplicative. For any multiplicative process, the time-average growth rate is approximately:

$$g \approx \mu - \frac{\sigma^2}{2}$$

where μ is the arithmetic mean return and σ^2 is the variance. The term $\sigma^2/2$ is the **volatility tax** — the price paid for compounding through volatile returns. The higher the volatility, the larger the wedge between what a portfolio *earns* (arithmetic mean) and what an investor *keeps* (geometric mean). Under fat tails, the ruin probability increases even when median outcomes look similar, because the left tail contains catastrophic draws that destroy compounding paths. A process is *ergodic* if its time average equals its ensemble average. Multiplicative processes with fat tails are non-ergodic: the typical (median) outcome diverges from the expected (mean) outcome. Most investors experience the time average, not the ensemble average (Peters, 2019).

The **Barbell strategy** (Taleb, *Antifragile*) is the most direct response to this problem. Allocate ~85% to the safest possible asset (T-bills) and ~15% to maximally aggressive bets. The result: bounded downside with unbounded upside potential. It structurally avoids catastrophic drawdowns — the maximum possible loss is bounded by the risky sleeve allocation. It does not require accurate tail parameter estimation (unlike CVaR optimization) or liquid options markets (unlike tail insurance). The advantage comes entirely from minimizing the volatility tax: by capping drawdowns, the Barbell preserves geometric compounding even when the aggressive sleeve experiences extreme losses.

The trade-off is real: the barbell underperforms in long bull markets where moderate risk is consistently rewarded. The question is whether that cost is worth the structural protection — and whether the answer changes when we look beyond the US.

```
In [2]: # === Simulation: The Volatility Tax Under Fat Tails ===
np.random.seed(42)

n_paths = 50_000
n_months = 360
mu_monthly = 0.10 / 12 # All four assets: same 10% annual arithmetic return

# Three Gaussian assets with SAME mean, DIFFERENT volatility
sigma_safe = 0.10 / np.sqrt(12)
sigma_market = 0.20 / np.sqrt(12)
sigma_conc = 0.40 / np.sqrt(12)

ret_safe = np.random.normal(loc=mu_monthly, scale=sigma_safe, size=(n_paths, n_months))
ret_market = np.random.normal(loc=mu_monthly, scale=sigma_market, size=(n_paths, n_months))
ret_conc = np.random.normal(loc=mu_monthly, scale=sigma_conc, size=(n_paths, n_months))

# Fat-tailed market: Student-t df=3, matched to same 20% annual vol
fat_scale = sigma_market / np.sqrt(3)
ret_fat = stats.t.rvs(df=3, loc=mu_monthly, scale=fat_scale, size=(n_paths, n_months))

tw_safe = np.prod(1 + ret_safe, axis=1)
tw_market = np.prod(1 + ret_market, axis=1)
tw_conc = np.prod(1 + ret_conc, axis=1)
tw_fat = np.prod(1 + ret_fat, axis=1)

assets = {
    'Safe (10% vol)': (tw_safe, 'steelblue'),
```

```

'Market (20% vol)': (tw_market, 'darkorange'),
'Fat-Tailed Market (t3, 20% vol)': (tw_fat, 'firebrick'),
'Concentrated (40% vol)': (tw_conc, 'dimgrey'),
}

print('=== Terminal Wealth After 30 Years: Same 10% Arithmetic Mean, Different Risk ===')
print(f'{"Asset":>40} {"Mean":>14} {"Median":>14} {"Mean/Med":>10} {"Geo CAGR":>10} {"Vol Tax":>10}')
print('-' * 108)
for label, (tw, _) in assets.items():
    mn, md = tw.mean(), np.median(tw)
    geo = (md ** (1/30) - 1) * 100
    vol_tax = 10.0 - geo
    ruin_pct = 100 * (tw < 1).mean()
    print(f'{"label":>40} ${{mn:>12,.0f}} ${{md:>12,.0f}} {{mn/md:>10.1f}} {{geo:>9.1f}}% {{vol_tax:>9.1f}}%')

fig, ax = plt.subplots(figsize=(14, 6))
bins = np.linspace(-6, 12, 150)
for label, (tw, color) in assets.items():
    log_tw = np.log(np.clip(tw, 1e-10, None))
    ax.hist(log_tw, bins=bins, alpha=0.35, color=color, density=True, label=label)
    ax.axvline(x=np.log(np.median(tw)), color=color, ls='--', lw=2)

ax.axvline(x=0, color='black', ls=':', lw=1, alpha=0.5, label='$1 (breakeven)')
ax.set_xlabel('log(Terminal Wealth)', fontsize=11)
ax.set_ylabel('Density', fontsize=11)
ax.set_title('Same 10% Arithmetic Mean, Different Tails \u2014 The Volatility Tax', fontsize=12)
ax.legend(fontsize=8, loc='upper right')
ax.set_yscale('log')
plt.tight_layout()
plt.show()

print('\nAll four assets earn 10% arithmetic mean. But the typical investor (median)')
print('earns dramatically less as volatility increases. Fat tails further increase ruin')
print('probability even when median outcomes look similar to the Gaussian case.')

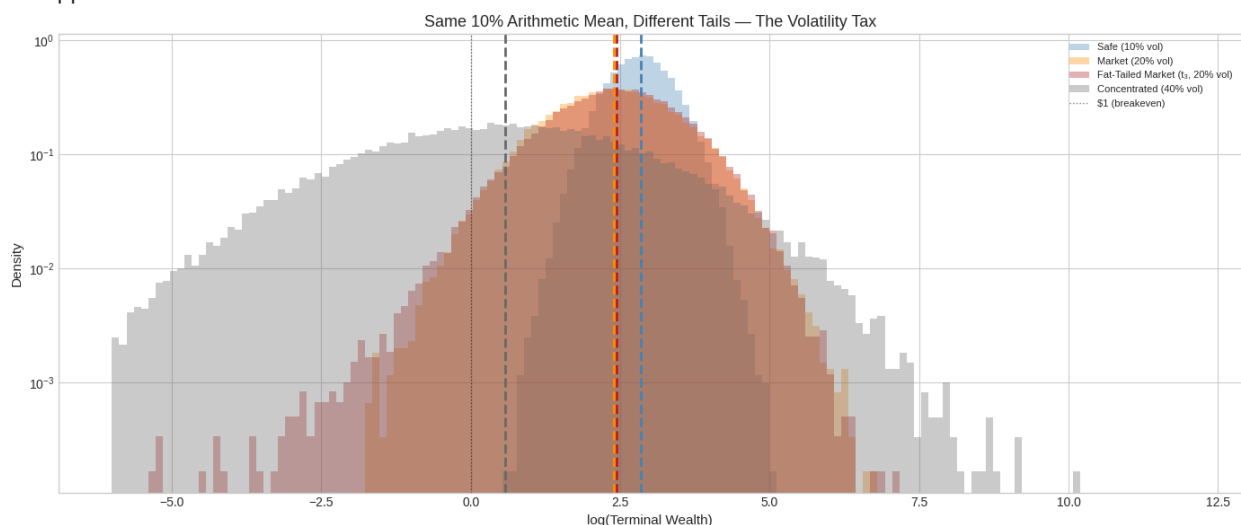
```

```

=== Terminal Wealth After 30 Years: Same 10% Arithmetic Mean, Different Risk ===

```

R	Vol Tax	Ruin %	Asset	Mean	Median	Mean/Med	Geo CAGR
0.1pp	0.0%		Safe (10% vol) \$	20 \$	17	1.2	9.9%
1.7pp	1.5%		Market (20% vol) \$	20 \$	11	1.8	8.3%
1.5pp	3.3%		Fat-Tailed Market (t3, 20% vol) \$	20 \$	11	1.7	8.5%
8.1pp	40.0%		Concentrated (40% vol) \$	19 \$	2	10.5	1.9%



All four assets earn 10% arithmetic mean. But the typical investor (median) earns dramatically less as volatility increases. Fat tails further increase ruin probability even when median outcomes look similar to the Gaussian case.

In [3]: # === Download Fama-French Five Factors ===

```
def download_french_data(url=None, csvname=None, skiplines=None):
    response = requests.get(url)
    if response.status_code != 200:
        raise Exception(f"Failed to download. Status code: {response.status_code}")
    with zipfile.ZipFile(BytesIO(response.content)) as zip_file:
        if csvname in zip_file.namelist():
            with zip_file.open(csvname) as csv_file:
                lines = csv_file.readlines()
                lines = lines[skiplines:]
                lines = [line.decode("utf-8") for line in lines]
                while lines and len(lines[0].strip()) == 0:
                    lines = lines[1:]
                for idx, line in enumerate(lines):
                    if idx == 0:
                        continue
                    if ('Annual Factors' in line) or (len(line.strip()) == 0):
                        break
                lines = lines[:idx]
                clean_csv = '\n'.join(lines)
                df = pd.read_csv(StringIO(clean_csv))
                df['date'] = df['Unnamed: 0'].apply(lambda x: datetime.strptime(str(x).strip(), '%Y-%m'))
                df = df.drop(columns='Unnamed: 0')
                df = df.set_index('date') / 100
                print(f'File {csvname} read successfully!')
                return df
        else:
            print(f'File {csvname} not found in archive.')
            return pd.DataFrame()

url = 'https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/F-F_Research_Data_5_Factors_2x3.csv'
csvname = 'F-F_Research_Data_5_Factors_2x3.csv'
ff5 = download_french_data(url=url, csvname=csvname, skiplines=4)

ff5 = ff5.loc['1963-07:']
print(f'\nDate range: {ff5.index[0].strftime("%Y-%m")} to {ff5.index[-1].strftime("%Y-%m")}')
print(f'Observations: {len(ff5)}')
print(f'Columns: {list(ff5.columns)}')
ff5.head()
```

File F-F_Research_Data_5_Factors_2x3.csv read successfully!

Date range: 1963-07 to 2025-12

Observations: 750

Columns: ['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA', 'RF']

Out[3]:

	Mkt-RF	SMB	HML	RMW	CMA	RF
date						
1963-07-01	-0.0039	-0.0048	-0.0081	0.0064	-0.0115	0.0027
1963-08-01	0.0508	-0.0080	0.0170	0.0040	-0.0038	0.0025
1963-09-01	-0.0157	-0.0043	0.0000	-0.0078	0.0015	0.0027
1963-10-01	0.0254	-0.0134	-0.0004	0.0279	-0.0225	0.0029
1963-11-01	-0.0086	-0.0085	0.0173	-0.0043	0.0227	0.0027

In [4]: # === Download External Data: Gold Prices and 10-Year Treasury Yield ===

```
def download_fred_csv(series_id, start_date='1963-01-01', end_date='2025-12-31'):
    url = f'https://fred.stlouisfed.org/graph/fredgraph.csv?id={series_id}&cosd={start_date}'
    headers = {'User-Agent': 'Mozilla/5.0 (Python/requests)'}
    resp = requests.get(url, headers=headers)
    if resp.status_code != 200:
        raise Exception(f'FRED download failed for {series_id}: HTTP {resp.status_code}')
    df = pd.read_csv(StringIO(resp.text), na_values='.')
```

```

date_col = [c for c in df.columns if 'date' in c.lower()][0]
df[date_col] = pd.to_datetime(df[date_col])
df = df.rename(columns={date_col: 'date'}).dropna().set_index('date')
print(f' {series_id}: {len(df)} observations, {df.index[0].strftime("%Y-%m")} to {df.index[-1].strftime("%Y-%m")} ')
return df

# --- Gold ---
print('Downloading gold price data...')
gold_url = 'https://raw.githubusercontent.com/datasets/gold-prices/main/data/monthly.csv'
gold_raw = pd.read_csv(gold_url)
gold_raw['Date'] = pd.to_datetime(gold_raw['Date'])
gold_raw = gold_raw.rename(columns={'Date': 'date', 'Price': 'gold_price'})
gold_raw = gold_raw.dropna().set_index('date')
gold_raw = gold_raw.loc['1963-01':]
gold_ret = gold_raw['gold_price'].pct_change().dropna()
gold_ret.name = 'gold_ret'
print(f' Gold returns: {gold_ret.index[0].strftime("%Y-%m")} to {gold_ret.index[-1].strftime("%Y-%m")} ')

# --- 10-Year Treasury Yield (GS10) from FRED ---
print('Downloading FRED GS10...')
gs10_raw = download_fred_csv('GS10', start_date='1963-01-01')
gs10_raw.columns = ['gs10']

gs10_monthly = gs10_raw['gs10'].resample('MS').last().dropna()
duration = 8.0
gs10_yield = gs10_monthly / 100
delta_yield = gs10_yield.diff()
tsy_ret = gs10_yield.shift(1) / 12 - duration * delta_yield
tsy_ret = tsy_ret.dropna()
tsy_ret.name = 'tsy_ret'
print(f' Treasury returns: {tsy_ret.index[0].strftime("%Y-%m")} to {tsy_ret.index[-1].strftime("%Y-%m")} ')

gold_ret.index = gold_ret.index.to_period('M').to_timestamp()
tsy_ret.index = tsy_ret.index.to_period('M').to_timestamp()

fred_data = pd.DataFrame({'gold_ret': gold_ret, 'tsy_ret': tsy_ret})
fred_data = fred_data.dropna()
print(f'\nExternal data aligned: {fred_data.index[0].strftime("%Y-%m")} to {fred_data.index[-1].strftime("%Y-%m")} ')
print(f'Gold ann. return: {gold_ret.mean()*1200:.2f}%, Treasury ann. return: {tsy_ret.mean()*1200:.2f}%')

```

Downloading gold price data...
 Gold returns: 1963-02 to 2025-07 (750 months)
 Downloading FRED GS10...
 GS10: 756 observations, 1963-01 to 2025-12
 Treasury returns: 1963-02 to 2025-12 (755 months)

 External data aligned: 1963-02 to 2025-07 (750 months)
 Gold ann. return: 8.53%, Treasury ann. return: 5.82%

Portfolio Definitions

#	Portfolio	Construction	Rationale
1	60/40	60% market + 40% T-bills	The default diversified allocation
2	80/20	80% market + 20% T-bills	More aggressive traditional
3	Five-Factor	Rolling 60-month Sharpe-maximized weights on FF5 factors	Sharpe-optimal benchmark (Topic 5 approach)
4	Barbell	85% T-bills + 15% convex risky sleeve	Bounded downside, convex upside
5	Kelly Barbell	Half-Kelly sized risky allocation (capped 30%), same sleeve	Adaptive barbell using trailing data

The risky sleeve (shared by portfolios 4-5): 40% 2x leveraged market, 25% long-term Treasuries, 15% gold, 20% crash insurance (Black-Scholes priced puts at 2σ OTM on trailing 60-month vol). All components use external data with no look-ahead bias.

In [5]: `# === Construct Five US Portfolios ===`

```
from scipy.stats import norm as norm_dist

factors = ['Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA']

# --- 1. 60/40 ---
port_6040 = 0.6 * ff5['Mkt-RF'] + ff5['RF']

# --- 2. 80/20 ---
port_8020 = 0.8 * ff5['Mkt-RF'] + ff5['RF']

# --- 3. Five-Factor Smart Beta ---
def neg_sharpe(weights, returns_df):
    port_ret = returns_df[factors].values @ weights
    if port_ret.std() == 0:
        return 0
    return -(np.sqrt(12) * port_ret.mean() / port_ret.std())

window = 60
smart_returns = []
w0 = np.array([0.2] * 5)
bounds = [(0, 0.5)] * 5
cons = {'type': 'ineq', 'fun': lambda w: 1.0 - np.sum(w)}

print(f'Running rolling optimization ({len(ff5) - window} months)...')
for t in range(window, len(ff5)):
    train = ff5.iloc[t-window:t]
    result = minimize(neg_sharpe, w0, args=(train,), method='SLSQP', bounds=bounds, constraints=cons)
    w_opt = result.x
    next_row = ff5.iloc[t]
    ret = np.dot(w_opt, next_row[factors].values) + next_row['RF']
    smart_returns.append({'date': ff5.index[t], 'return': ret})
    if (t - window) % 100 == 0:
        print(f'  Month {t - window}/{len(ff5) - window}')

port_smart = pd.DataFrame(smart_returns).set_index('date')['return']
print(f'  Done. {len(port_smart)} months of smart beta returns.')

# --- 4. Convex Barbell ---
lev_mkt = 2 * ff5['Mkt-RF'] + ff5['RF']
tsy_aligned = fred_data['tsy_ret'].reindex(ff5.index)
gold_aligned = fred_data['gold_ret'].reindex(ff5.index)

window_ins = 60
insurance_ret = pd.Series(np.nan, index=ff5.index)

print('Computing crash insurance returns...')
for t in range(window_ins, len(ff5)):
    trailing_mkt = ff5['Mkt-RF'].iloc[t-window_ins:t] + ff5['RF'].iloc[t-window_ins:t]
    sigma = trailing_mkt.std()
    rf = ff5['RF'].iloc[t]
    K = np.exp(-2 * sigma)
    if sigma > 1e-8:
        d1 = (-np.log(K) + rf + 0.5 * sigma**2) / sigma
        d2 = d1 - sigma
        premium = K * np.exp(-rf) * norm_dist.cdf(-d2) - norm_dist.cdf(-d1)
    else:
        premium = 0.0
    premium = max(premium, 1e-6)
    mkt_ret = ff5['Mkt-RF'].iloc[t] + ff5['RF'].iloc[t]
    realized_price = 1 + mkt_ret
    payout = max(K - realized_price, 0)
    insurance_ret.iloc[t] = payout - premium

print(f'  Insurance returns computed for {insurance_ret.dropna().shape[0]} months.')

risky_sleeve = (0.40 * lev_mkt +
                0.25 * tsy_aligned.fillna(0) +
```

```

        0.15 * gold_aligned.fillna(0) +
        0.20 * insurance_ret.fillna(0))

safe_frac = 0.85
risky_frac = 0.15
port_barbell = safe_frac * ff5['RF'] + risky_frac * risky_sleeve

# --- 5. Kelly Barbell ---
kelly_cap = 0.30
kelly_returns = []

print('Computing Kelly Barbell returns...')
for t in range(window_ins, len(ff5)):
    trailing = risky_sleeve.iloc[max(0, t-60):t].dropna()
    if len(trailing) < 12:
        kelly_returns.append({'date': ff5.index[t], 'return': ff5['RF'].iloc[t], 'risky_pct': 0})
        continue
    train_rf = ff5['RF'].iloc[max(0, t-60):t].reindex(trailing.index)
    excess_sleeve = trailing - train_rf
    mu = excess_sleeve.mean()
    sigma2 = excess_sleeve.var()
    if sigma2 > 1e-10 and mu > 0:
        f_star = 0.5 * mu / sigma2
    else:
        f_star = 0.0
    f_kelly = np.clip(f_star, 0.0, kelly_cap)
    ret = f_kelly * risky_sleeve.iloc[t] + (1 - f_kelly) * ff5['RF'].iloc[t]
    kelly_returns.append({'date': ff5.index[t], 'return': ret, 'risky_pct': f_kelly})

port_kelly = pd.DataFrame(kelly_returns).set_index('date')['return']
kelly_alloc_series = pd.DataFrame(kelly_returns).set_index('date')['risky_pct']
print(f' Kelly Barbell: {len(port_kelly)} months, median alloc = {kelly_alloc_series.median()}')

# --- Align ---
common_idx = port_smart.index
portfolios = pd.DataFrame({
    '60/40': port_6040.loc[common_idx],
    '80/20': port_8020.loc[common_idx],
    'Five-Factor': port_smart,
    'Barbell': port_barbell.loc[common_idx],
    'Kelly Barbell': port_kelly.loc[common_idx],
})
portfolios = portfolios.dropna()
risky_sleeve_aligned = risky_sleeve.loc[portfolios.index]

print(f'\nPortfolio date range: {portfolios.index[0].strftime("%Y-%m")} to {portfolios.index[-1].strftime("%Y-%m")}')
print(f'Observations: {len(portfolios)}')

print('\n=== Risky Sleeve Component Summary (annualized) ===')
components = {'Lev. Market': lev_mkt, 'Treasuries': tsy_aligned,
              'Gold': gold_aligned, 'Insurance': insurance_ret}
for cname, cseries in components.items():
    c = cseries.loc[portfolios.index].dropna()
    if len(c) > 0:
        print(f' {cname:15s}: mean {c.mean()*1200:7.2f}%, vol {c.std()*np.sqrt(12)*100:7.2f}%')

```

Running rolling optimization (690 months)...

Month 0/690
Month 100/690
Month 200/690
Month 300/690
Month 400/690
Month 500/690
Month 600/690

Done. 690 months of smart beta returns.

Computing crash insurance returns...

Insurance returns computed for 690 months.

Computing Kelly Barbell returns...

Kelly Barbell: 690 months, median alloc = 30.0%

Portfolio date range: 1968-07 to 2025-12

Observations: 690

=== Risky Sleeve Component Summary (annualized) ===

Lev. Market	: mean	18.45%,	vol	31.56%,	n=690
Treasuries	: mean	6.17%,	vol	8.12%,	n=685
Gold	: mean	9.06%,	vol	16.76%,	n=685
Insurance	: mean	1.00%,	vol	3.11%,	n=690

In [6]: # === Summary Statistics + Cumulative Wealth ===

```
summary = pd.DataFrame({
    'Ann. Mean (%)': portfolios.mean() * 1200,
    'Ann. Std (%)': portfolios.std() * np.sqrt(12) * 100,
    'Min Monthly (%)': portfolios.min() * 100,
    'Max Monthly (%)': portfolios.max() * 100,
}, index=portfolios.columns).round(2)

print('=== Portfolio Summary Statistics ===')
print(summary.to_string())
print('\n=== Correlation Matrix ===')
print(portfolios.corr().round(3).to_string())

wealth = (1 + portfolios).cumprod()

fig, ax = plt.subplots(figsize=(14, 7))
for name in portfolios.columns:
    ax.plot(wealth.index, wealth[name], color=COLORS[name], lw=1.5, label=name)

ax.set_yscale('log')
ax.set_ylabel('Cumulative Wealth (log scale)', fontsize=12)
ax.set_title('Cumulative Wealth: $1 Invested', fontsize=14)
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

for name in portfolios.columns:
    final = wealth[name].iloc[-1]
    ax.annotate(f'${final:,.0f}', xy=(wealth.index[-1], final),
               xytext=(10, 0), textcoords='offset points',
               fontsize=10, color=COLORS[name], fontweight='bold')

plt.tight_layout()
plt.show()

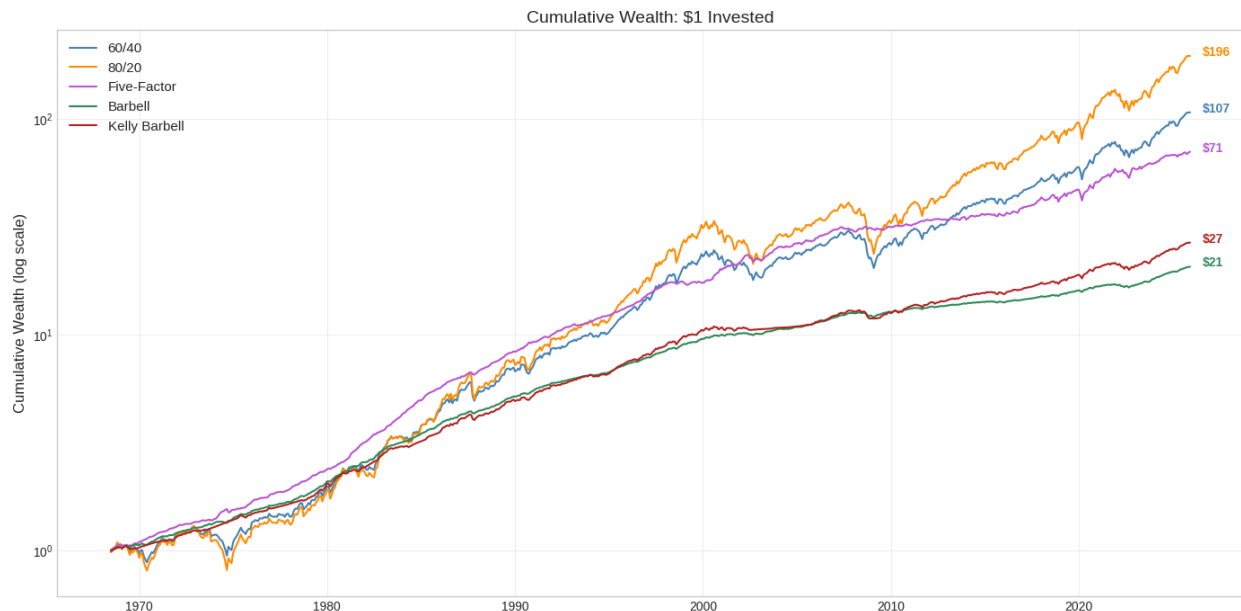
n_years = len(portfolios) / 12
print(f'\n=== Final Wealth and CAGR ({n_years:.1f} years) ===')
for name in portfolios.columns:
    final = wealth[name].iloc[-1]
    cagr = final ** (1 / n_years) - 1
    print(f' {name:15s}: ${final:>10,.2f} CAGR = {cagr:.2%}')
```

=== Portfolio Summary Statistics ===

	Ann. Mean (%)	Ann. Std (%)	Min Monthly (%)	Max Monthly (%)
60/40	8.61	9.46	-13.31	10.17
80/20	10.01	12.61	-17.95	13.39
Five-Factor	7.51	4.05	-6.51	6.86
Barbell	5.30	2.08	-1.78	2.57
Kelly Barbell	5.79	3.38	-4.15	3.97

=== Correlation Matrix ===

	60/40	80/20	Five-Factor	Barbell	Kelly Barbell
60/40	1.000	1.000	0.377	0.914	0.833
80/20	1.000	1.000	0.372	0.906	0.829
Five-Factor	0.377	0.372	1.000	0.401	0.419
Barbell	0.914	0.906	0.401	1.000	0.858
Kelly Barbell	0.833	0.829	0.419	0.858	1.000



```

=== Final Wealth and CAGR (57.5 years) ===
60/40      : $    107.27  CAGR = 8.47%
80/20      : $    195.90  CAGR = 9.61%
Five-Factor : $     70.51  CAGR = 7.68%
Barbell    : $     20.66  CAGR = 5.41%
Kelly Barbell : $     26.74  CAGR = 5.88%

```

```
In [7]: # === Performance Metrics + Distribution Tests ===
```

```

rf_aligned = ff5.loc[portfolios.index, 'RF']
excess = portfolios.subtract(rf_aligned, axis=0)

metrics = {}
for name in portfolios.columns:
    ex = excess[name]
    downside = ex[ex < 0]
    downside_std = downside.std() if len(downside) > 0 else 0.001
    metrics[name] = {
        'Arith. Mean (ann. %)': ex.mean() * 1200,
        'Volatility (ann. %)': ex.std() * np.sqrt(12) * 100,
        'Sharpe Ratio': np.sqrt(12) * ex.mean() / ex.std() if ex.std() > 0 else 0,
        'Sortino Ratio': np.sqrt(12) * ex.mean() / downside_std if downside_std > 0 else 0,
    }

metrics_df = pd.DataFrame(metrics).T.round(3)
print('=== Performance Metrics ===')
print(metrics_df.to_string())

best_sharpe = metrics_df['Sharpe Ratio'].idxmax()
print(f'\nHighest Sharpe: {best_sharpe} ({metrics_df.loc[best_sharpe, "Sharpe Ratio"]:.3f})')
print('But Sharpe is a variance-based metric that underestimates risk under fat tails.')

dist_stats = {}
for name in portfolios.columns:
    r = portfolios[name].values
    jb_stat, jb_p, skew_jb, kurt_jb = jarque_bera(r)
    dist_stats[name] = {
        'Skewness': stats.skew(r),
        'Excess Kurtosis': stats.kurtosis(r),
        'Jarque-Bera Stat': jb_stat,
        'JB p-value': jb_p,
    }

dist_df = pd.DataFrame(dist_stats).T.round(4)
print('\n=== Distribution Diagnostics ===')
print(dist_df.to_string())
print('\nAll five portfolios reject Gaussianity (JB p < 0.05).')

```

=== Performance Metrics ===

	Arith. Mean (ann. %)	Volatility (ann. %)	Sharpe Ratio	Sortino Ratio
60/40	4.217	9.487	0.445	0.625
80/20	5.623	12.649	0.445	0.625
Five-Factor	3.117	3.972	0.785	0.996
Barbell	0.910	1.963	0.464	0.689
Kelly Barbell	1.396	3.353	0.416	0.537

Highest Sharpe: Five-Factor (0.785)

But Sharpe is a variance-based metric that underestimates risk under fat tails.

=== Distribution Diagnostics ===

	Skewness	Excess Kurtosis	Jarque-Bera Stat	JB p-value
60/40	-0.4818	1.6139	101.5749	0.0
80/20	-0.4878	1.6180	102.6370	0.0
Five-Factor	-0.4465	5.6386	937.0135	0.0
Barbell	-0.0091	0.8362	20.1120	0.0
Kelly Barbell	-0.3371	1.7925	105.4446	0.0

All five portfolios reject Gaussianity (JB p < 0.05).

In [8]: # === Drawdown Analysis ===

```
wealth = (1 + portfolios).cumprod()
running_max = wealth.cummax()
drawdown = (wealth - running_max) / running_max

ulcer_index = np.sqrt((drawdown ** 2).mean())
ann_excess_mean = excess.mean() * 12
martin_ratio = ann_excess_mean / ulcer_index

pain_df = pd.DataFrame({
    'Ulcer Index (%)': (ulcer_index * 100).round(2),
    'Martin Ratio': martin_ratio.round(3),
    'Max Drawdown (%)': (drawdown.min() * 100).round(2),
})
print('=== Pain-Adjusted Metrics ===')
print(pain_df.to_string())

fig, ax = plt.subplots(figsize=(14, 5))
for name in portfolios.columns:
    dd = drawdown[name]
    ax.fill_between(dd.index, dd * 100, 0, alpha=0.3, color=COLORS[name])
    ax.plot(dd.index, dd * 100, color=COLORS[name], lw=0.8, label=name)
ax.set_ylabel('Drawdown (%)')
ax.set_title('Underwater Plot: Drawdown History', fontsize=14)
ax.legend(loc='lower left')
ax.yaxis.set_major_formatter(PercentFormatter(decimals=0))
plt.tight_layout()
plt.show()

def get_top_drawdowns(returns, n=5):
    w = (1 + returns).cumprod()
    rm = w.cummax()
    dd = (w - rm) / rm
    in_dd = dd < 0
    starts = in_dd & ~in_dd.shift(1, fill_value=False)
    ends = ~in_dd & in_dd.shift(1, fill_value=False)
    s_dates = dd.index[starts].tolist()
    e_dates = dd.index[ends].tolist()
    if len(e_dates) < len(s_dates):
        e_dates.append(dd.index[-1])
    records = []
    for i, s in enumerate(s_dates):
        e = e_dates[i] if i < len(e_dates) and e_dates[i] > s else dd.index[-1]
        seg = dd.loc[s:e]
        trough = seg.idxmin()
        depth = seg.min()
        dur = len(seg)
```

```

        records.append({'Start': s.strftime('%Y-%m'), 'Trough': trough.strftime('%Y-%m'),
                        'Depth': depth, 'Depth (%)': f'{depth*100:.1f}', 'Duration (mo)': d
        return pd.DataFrame(records).sort_values('Depth').head(n).drop(columns='Depth').reset_

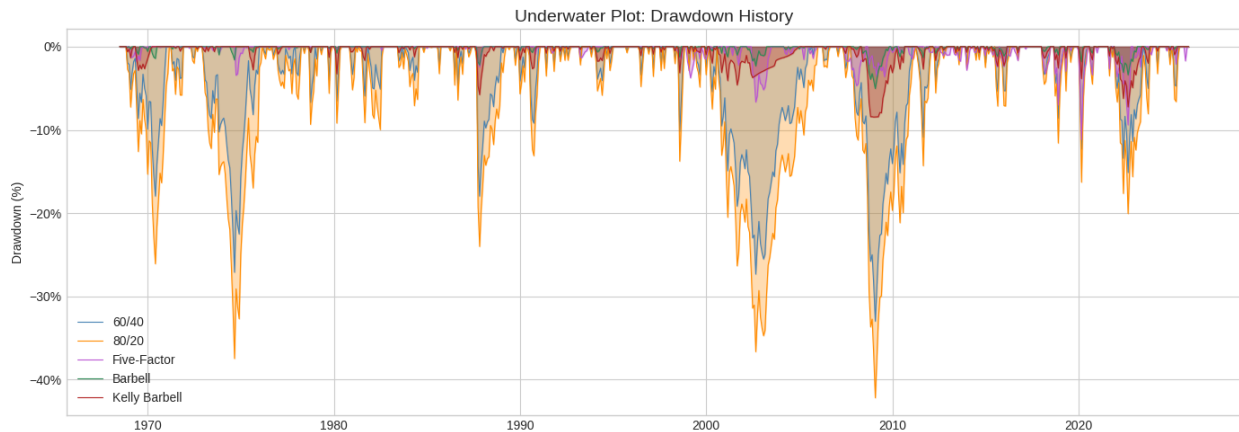
for name in portfolios.columns:
    print(f'\n--- Worst Drawdowns: {name} ---')
    print(get_top_drawdowns(portfolios[name]).to_string(index=False))

print("\nThe Barbell's drawdowns are structurally capped by the 85% T-bill allocation.")

```

=== Pain-Adjusted Metrics ===

	Ulcer Index (%)	Martin Ratio	Max Drawdown (%)
60/40	6.94	0.608	-32.97
80/20	9.97	0.564	-42.18
Five-Factor	1.30	2.399	-11.18
Barbell	0.59	1.546	-5.04
Kelly Barbell	1.54	0.909	-8.46



--- Worst Drawdowns: 60/40 ---

Start	Trough	Depth (%)	Duration (mo)
2007-11	2009-02	-33.0	40
2000-09	2002-09	-27.3	59
1973-01	1974-09	-27.1	37
1987-09	1987-11	-18.0	17
1968-12	1970-06	-17.9	26

--- Worst Drawdowns: 80/20 ---

Start	Trough	Depth (%)	Duration (mo)
2007-11	2009-02	-42.2	42
1973-01	1974-09	-37.5	39
2000-09	2002-09	-36.6	65
1968-12	1970-06	-26.1	28
1987-09	1987-11	-24.0	20

--- Worst Drawdowns: Five-Factor ---

Start	Trough	Depth (%)	Duration (mo)
2020-01	2020-03	-11.2	7
2022-01	2022-09	-9.4	11
2018-10	2018-12	-7.0	7
2002-06	2002-09	-6.7	14
2007-07	2008-02	-4.7	14

--- Worst Drawdowns: Barbell ---

Start	Trough	Depth (%)	Duration (mo)
2008-06	2009-02	-5.0	18
2022-01	2022-09	-3.4	15
2002-04	2002-09	-2.3	14
1987-09	1987-11	-2.2	6
2020-02	2020-03	-1.8	4

--- Worst Drawdowns: Kelly Barbell ---

Start	Trough	Depth (%)	Duration (mo)
2008-06	2009-01	-8.5	28
2022-01	2022-09	-7.3	18
1987-09	1987-11	-5.8	10
2000-09	2001-09	-4.6	53
2020-02	2020-03	-3.8	4

The Barbell's drawdowns are structurally capped by the 85% T-bill allocation.

```
In [9]: # === Geometric Growth Rate and Volatility Tax ===

geo_growth = 12 * np.log(1 + portfolios).mean()
arith_growth = 12 * portfolios.mean()
vol_tax = arith_growth - geo_growth

growth_df = pd.DataFrame({
    'Arithmetic Growth (ann. %)': (arith_growth * 100).round(3),
    'Geometric Growth (ann. %)': (geo_growth * 100).round(3),
    'Volatility Tax (ann. %)': (vol_tax * 100).round(3),
})
print('=== Growth Rates and Volatility Tax ===')
print(growth_df.to_string())
print('\nThe Barbell minimizes the volatility tax but at a cost in geometric growth')
print('that US data does not justify on its own. Part 2 asks whether non-US data changes th
```

```
=== Growth Rates and Volatility Tax ===
```

	Arithmetic Growth (ann. %)	Geometric Growth (ann. %)	Volatility Tax (ann. %)
60/40	8.606	8.131	0.4
75			
80/20	10.012	9.178	0.8
34			
Five-Factor	7.506	7.401	0.1
05			
Barbell	5.300	5.267	0.0
33			
Kelly Barbell	5.786	5.715	0.0
70			

The Barbell minimizes the volatility tax but at a cost in geometric growth
that US data does not justify on its own. Part 2 asks whether non-US data changes this calc
ulus.

```
In [10]: # === Monte Carlo Simulation + Horizon Sweep ===

np.random.seed(42)
n_simulations = 10_000
n_months_mc = 360

terminal_wealth = {}
for name in portfolios.columns:
    returns = portfolios[name].values
    simulated = np.random.choice(returns, size=(n_simulations, n_months_mc), replace=True)
    terminal_wealth[name] = np.prod(1 + simulated, axis=1)

tw = pd.DataFrame(terminal_wealth)

fig, ax = plt.subplots(figsize=(14, 6))
for name in portfolios.columns:
    log_tw = np.log10(np.clip(tw[name].values, 1e-3, None))
    ax.hist(log_tw, bins=80, alpha=0.35, color=COLORS[name], density=True, label=name)
    ax.axvline(x=np.log10(tw[name].median()), color=COLORS[name], ls='--', lw=2)
```

```

ax.set_xlabel('Terminal Wealth (log scale)', fontsize=12)
ax.set_ylabel('Density', fontsize=12)
ax.set_title('Monte Carlo: 30-Year Terminal Wealth Distribution (10,000 paths)', fontsize=12)
ax.legend(fontsize=11)
plt.tight_layout()
plt.show()

mc_summary = pd.DataFrame({
    'Mean Wealth ($)': tw.mean().round(0),
    'Median Wealth ($)': tw.median().round(0),
    'Mean/Median': (tw.mean() / tw.median()).round(2),
    'P5 ($)': tw.quantile(0.05).round(0),
    'P95 ($)': tw.quantile(0.95).round(0),
    'P(Wealth < $1)': (tw < 1).mean().round(4),
})
print('=== Monte Carlo: 30-Year Terminal Wealth ===')
print(mc_summary.to_string())

print('\n--- Ergodicity Analysis ---')
for name in portfolios.columns:
    ratio = tw[name].mean() / tw[name].median()
    print(f'{name}: Mean/Median ratio = {ratio:.2f}')

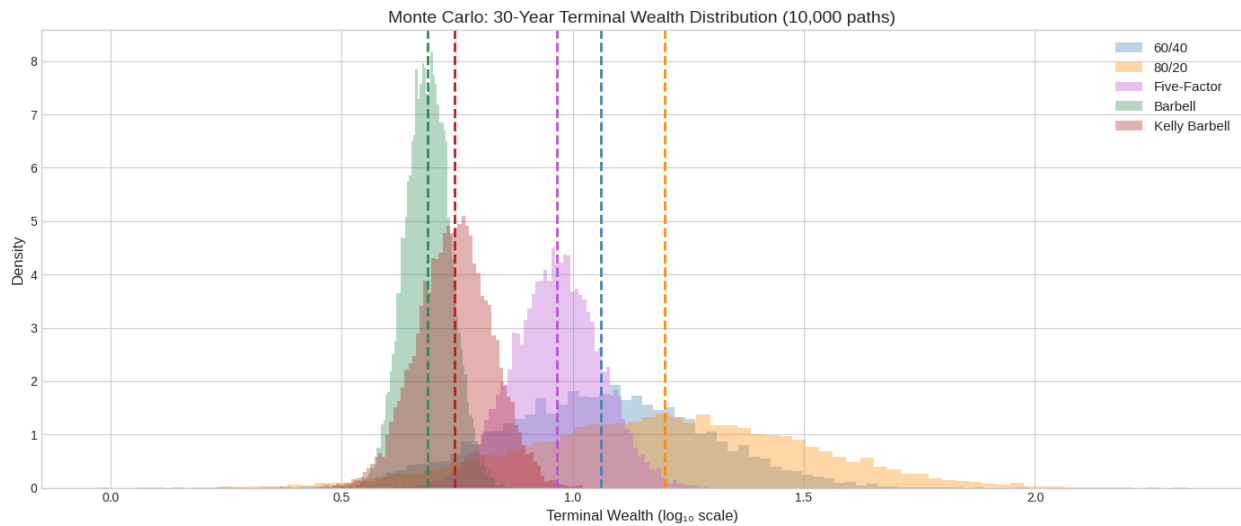
# Horizon sweep
print('\n=== Horizon Sweep: Median Terminal Wealth by Investment Horizon ===')
horizons = [10, 20, 30, 40, 50]
horizon_results = {name: [] for name in portfolios.columns}

np.random.seed(42)
for horizon in horizons:
    n_months_h = horizon * 12
    for name in portfolios.columns:
        returns = portfolios[name].values
        simulated = np.random.choice(returns, size=(n_simulations, n_months_h), replace=True)
        median_tw = np.median(np.prod(1 + simulated, axis=1))
        horizon_results[name].append(median_tw)

print(f'{"Horizon":>10}', end='')
for name in portfolios.columns:
    print(f'{name:>15}', end='')
print()
for i, h in enumerate(horizons):
    print(f'{h:>8} yr', end='')
    for name in portfolios.columns:
        print(f' ${horizon_results[name][i]:>11,.0f}', end='')
    print()

fig, ax = plt.subplots(figsize=(12, 6))
for name in portfolios.columns:
    ax.plot(horizons, horizon_results[name], 'o-', color=COLORS[name],
            lw=2, markersize=8, label=name)
ax.set_xlabel('Investment Horizon (years)', fontsize=12)
ax.set_ylabel('Median Terminal Wealth ($)', fontsize=12)
ax.set_title('Median Terminal Wealth by Horizon (10,000 Monte Carlo paths)', fontsize=14)
ax.set_yscale('log')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



=== Monte Carlo: 30-Year Terminal Wealth ===

	Mean Wealth (\$)	Median Wealth (\$)	Mean/Median	P5 (\$)	P95 (\$)	P(Wealth < \$1)
60/40	13.0	12.0	1.14	5.0	27.0	0.0000
80/20	20.0	16.0	1.27	5.0	49.0	0.0002
Five-Factor	9.0	9.0	1.02	6.0	13.0	0.0000
Barbell	5.0	5.0	1.01	4.0	6.0	0.0000
Kelly Barbell	6.0	6.0	1.02	4.0	7.0	0.0000

--- Ergodicity Analysis ---

60/40: Mean/Median ratio = 1.14

80/20: Mean/Median ratio = 1.27

Five-Factor: Mean/Median ratio = 1.02

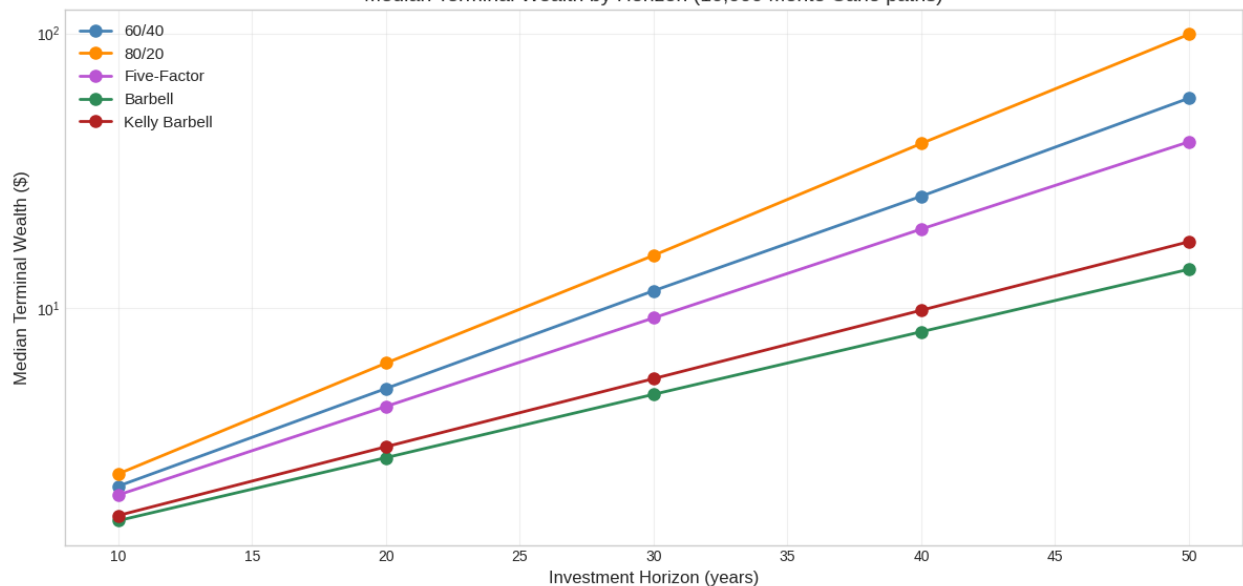
Barbell: Mean/Median ratio = 1.01

Kelly Barbell: Mean/Median ratio = 1.02

=== Horizon Sweep: Median Terminal Wealth by Investment Horizon ===

Horizon	60/40	80/20	Five-Factor	Barbell	Kelly Barbell
10 yr \$	2 \$	3 \$	2 \$	2 \$	2
20 yr \$	5 \$	6 \$	4 \$	3 \$	3
30 yr \$	12 \$	16 \$	9 \$	5 \$	6
40 yr \$	26 \$	40 \$	19 \$	8 \$	10
50 yr \$	58 \$	99 \$	40 \$	14 \$	17

Median Terminal Wealth by Horizon (10,000 Monte Carlo paths)



```
In [11]: # === Retirement Outcomes: DCA + Lump Sum ===
```

```
annual_salary = 85_000
savings_rate = 0.15
monthly_contribution = annual_salary * savings_rate / 12
annual_inflation = 0.03
monthly_inflation = (1 + annual_inflation) ** (1/12) - 1

target_adequate = 400_000
target_comfortable = 750_000

strat_returns = {}
for name in portfolios.columns:
    strat_returns[name] = (1 + portfolios[name]) / (1 + monthly_inflation) - 1

n_months = len(portfolios)
dates = portfolios.index

# --- DCA: 30-year careers ---
career_years = 30
career_months = career_years * 12
max_starts = n_months - career_months

results = {}
for sname, ret_series in strat_returns.items():
    rv = ret_series.values
    tw_list = []
    for s in range(max_starts):
        w = 0.0
        for m in range(career_months):
            w = w * (1 + rv[s + m]) + monthly_contribution
        tw_list.append(w)
    results[sname] = np.array(tw_list)

start_dates = dates[:max_starts]

print(f'=== DCA Retirement Wealth ({career_years}-year career, ${monthly_contribution:,.2f} / month) ===')
print(f'{"Strategy":>15} {"Median":>12} {"P10":>12} {"P90":>12} {"\u2265$400K":>8} {"\u2265$750K":>8}')
print('-' * 68)
for sname, tw_arr in results.items():
    pct_400k = 100 * np.mean(tw_arr >= target_adequate)
    pct_750k = 100 * np.mean(tw_arr >= target_comfortable)
    print(f'{sname:>15} ${np.median(tw_arr):>10,.0f} ${np.percentile(tw_arr,10):>10,.0f} '
          f'${np.percentile(tw_arr,90):>10,.0f} {pct_400k:>6.1f}% {pct_750k:>6.1f}%')

# DCA violin
fig, ax = plt.subplots(figsize=(14, 7))
positions = np.arange(len(strat_returns))
violin_data = [results[name] / 1000 for name in strat_returns.keys()]
parts = ax.violinplot(violin_data, positions=positions, showmeans=True, showmedians=True,
                      showquartiles=True, showboxes=False)
for i, (pc, name) in enumerate(zip(parts['bodies'], strat_returns.keys())):
    pc.set_facecolor(COLORS[name])
    pc.set_alpha(0.6)
parts['cmeans'].set_color('black')
parts['cmmedians'].set_color('red')
ax.axhline(y=target_adequate/1000, color='red', ls=':', lw=1.5, alpha=0.7, label=f'${target_adequate/1000}K')
ax.axhline(y=target_comfortable/1000, color='gold', ls=':', lw=1.5, alpha=0.7, label=f'${target_comfortable/1000}K')
ax.set_xticks(positions)
ax.set_xticklabels(list(strat_returns.keys()), fontsize=11)
ax.set_ylabel('Terminal Wealth ($K, 2024 dollars)', fontsize=12)
ax.set_title(f'Retirement Wealth: $85K Income, 15% Savings, {career_years}-Year Career', fontweight='bold')
ax.legend(fontsize=10, loc='upper right')
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

# --- Lump Sum ---
print('\n' + '=' * 80)
print('LUMP SUM: $100,000 invested, 20-year horizon (real 2024$)')
```

```

print('=' * 80)

lump_sum_val = 100_000
lump_horizon = 20
lump_months = lump_horizon * 12
max_starts_lump = n_months - lump_months

lump_results = {}
lump_drawdowns = {}
for sname, ret_series in strat_returns.items():
    rv = ret_series.values
    terminals = []
    max_dd = []
    for s in range(max_starts_lump):
        cum_ret = np.cumprod(1 + rv[s:s + lump_months])
        path = lump_sum_val * cum_ret
        terminals.append(path[-1])
        running_max_path = np.maximum.accumulate(path)
        dd = (path - running_max_path) / running_max_path
        max_dd.append(dd.min())
    lump_results[sname] = np.array(terminals)
    lump_drawdowns[sname] = np.array(max_dd)

lump_dates = dates[:max_starts_lump]
print(f'\n{"Strategy":>15} {"Median":>12} {"P10":>12} {"P90":>12} {"Med MaxDD":>10} {"Worst"}')
print('-' * 73)
for sname in strat_returns:
    tw_arr = lump_results[sname]
    mdd = lump_drawdowns[sname]
    print(f'{sname:>15} ${np.median(tw_arr):>10,.0f} ${np.percentile(tw_arr, 10):>10,.0f}
          f'${np.percentile(tw_arr, 90):>10,.0f} {np.median(mdd):>9.1%} {np.min(mdd):>11.1%}')

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

ax = axes[0]
violin_data_l = [lump_results[name] / 1000 for name in strat_returns.keys()]
parts_l = ax.violinplot(violin_data_l, positions=positions, showmeans=True, showmedians=True)
for i, (pc, name) in enumerate(zip(parts_l['bodies'], strat_returns.keys())):
    pc.set_facecolor(COLORS[name])
    pc.set_alpha(0.6)
parts_l['cmeans'].set_color('black')
parts_l['cmmedians'].set_color('red')
ax.set_xticks(positions)
ax.set_xticklabels(list(strat_returns.keys()), fontsize=10)
ax.set_ylabel('Terminal Wealth ($K, 2024 dollars)', fontsize=11)
ax.set_title(f'Lump Sum $100K: {lump_horizon}-Year Terminal Wealth', fontsize=13)
ax.grid(True, alpha=0.3, axis='y')

ax = axes[1]
violin_dd = [-lump_drawdowns[name] * 100 for name in strat_returns.keys()]
parts_dd = ax.violinplot(violin_dd, positions=positions, showmeans=False, showmedians=True)
for i, (pc, name) in enumerate(zip(parts_dd['bodies'], strat_returns.keys())):
    pc.set_facecolor(COLORS[name])
    pc.set_alpha(0.6)
parts_dd['cmmedians'].set_color('red')
ax.set_xticks(positions)
ax.set_xticklabels(list(strat_returns.keys()), fontsize=10)
ax.set_ylabel('Max Drawdown (%)', fontsize=11)
ax.set_title(f'Worst Drawdown During {lump_horizon}-Year Hold', fontsize=13)
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

print(f"\nThe lump-sum analysis reveals the barbell's real value: drawdown control.")
print(f'For accumulating wealth through DCA, the higher-beta portfolios dominate on median')

```

=== DCA Retirement Wealth (30-year career, \$1,062.50/mo, real 2024\$) ===

Strategy	Median	P10	P90	≥\$400K	≥\$750K
----------	--------	-----	-----	---------	---------

60/40	\$ 752,127	\$ 671,592	\$ 1,569,157	100.0%	50.6%
80/20	\$ 947,421	\$ 838,804	\$ 2,063,180	100.0%	99.4%
Five-Factor	\$ 665,350	\$ 517,324	\$ 1,185,099	100.0%	43.6%
Barbell	\$ 468,420	\$ 381,565	\$ 756,457	65.8%	11.2%
Kelly Barbell	\$ 490,323	\$ 432,106	\$ 831,006	100.0%	14.8%



LUMP SUM: \$100,000 invested, 20-year horizon (real 2024\$)

Strategy	Median	P10	P90	Med MaxDD	Worst MaxDD
60/40	\$ 295,099	\$ 172,064	\$ 567,454	-31.7%	-35.7%
80/20	\$ 351,325	\$ 209,836	\$ 745,964	-40.7%	-45.5%
Five-Factor	\$ 266,060	\$ 140,490	\$ 448,128	-7.6%	-12.5%
Barbell	\$ 158,454	\$ 93,775	\$ 271,164	-5.8%	-15.3%
Kelly Barbell	\$ 170,513	\$ 107,827	\$ 282,302	-12.1%	-15.7%



The lump-sum analysis reveals the barbell's real value: drawdown control. For accumulating wealth through DCA, the higher-beta portfolios dominate on median outcome.

Part 1 Findings

- 80/20 dominates on terminal wealth in every US scenario.** Whether measured by CAGR, Monte Carlo median, or retirement adequacy, higher equity exposure compounds more effectively over a full career. The Five-Factor portfolio wins on Sharpe ratio but introduces factor-regime sensitivity that widens outcome dispersion.
- The Barbell trades ~400 bps of CAGR for predictability and bounded drawdowns.** Its T-bill floor

structurally limits max drawdown to about -5% versus the 80/20's -42% — roughly one-eighth the pain. The Kelly Barbell captures more upside through adaptive sizing (though it hits its 30% cap in most months, making it effectively a fixed 30/70 variant). The cost in terminal wealth is real and persistent.

3. **The choice is genuine but asymmetric — it depends on temperament.** Higher-beta portfolios win the compounding race. The barbell wins the sleep-at-night race. For a 30-year accumulator with a steady paycheck (DCA), the 80/20 is hard to argue against. For a lump-sum investor protecting existing wealth, the barbell's drawdown cap is structural, not estimated.

But there is a quiet assumption embedded in all three findings: the T-bill — the "safe" asset in every portfolio above — actually preserves value. In the United States, from 1963 to 2024, it did. But T-bills are government promises denominated in government currency. They are "safe" only as long as the institution behind them remains solvent and the currency retains purchasing power. The next section asks what happens when it doesn't.

"But wait a minute."

Every analysis above uses US data. The US equity market is the most successful in history — the country that won two world wars, avoided hyperinflation, maintained institutional continuity for 250 years, and emerged as the sole superpower. When I say "80/20 dominates in every historical scenario," I mean every scenario *from the one country where equities would have dominated*.

This is the survivorship bias problem in portfolio analysis. We optimize on the winner and call it universal. To genuinely test whether the barbell thesis holds, I need data from the countries where the conventional wisdom failed — where equities did not always recover, where bills did not always preserve value, where institutions collapsed and reformed.

If 80/20 still dominates everywhere, then Part 1's conclusion is robust and the barbell is an expensive insurance policy that never pays. If it doesn't, then the "right" portfolio depends on what you believe about the future of institutions — not on what a backtest tells you.

Part 2: The International Case

The Data: JST Macrohistory + Gold

The **Jordà-Schularick-Taylor Macrohistory Database** (Jordà, Schularick & Taylor, 2017) provides annual asset returns for up to 18 countries from 1870 to 2020. After filtering for complete data (equity returns, bill rates, and CPI), 16 countries remain. This includes Germany (hyperinflation, two world wars, partition, reunification), Japan (imperial expansion, defeat, reconstruction, lost decades), Italy, France, Spain, and other countries whose financial histories include regime breaks that the US was spared.

I add **gold** as an alternative safe asset — the asset that carries no counterparty risk and cannot be devalued by government decree. Gold's USD price was fixed under the gold standard, but its **local-currency price** moved with exchange rates. When Germany hyperinflated in the 1920s, the mark collapsed against the dollar, and gold's mark-denominated price skyrocketed — exactly the crisis protection gold is supposed to provide. I compute gold's local-currency return for each country-year as: USD gold price × local exchange rate (from JST's `xrusd` field), then deflate by local CPI. This correctly captures gold's role during regime breaks: gold preserved value in the countries that debased their currencies, even when the USD gold price was nominally fixed.

```
In [12]: # === Load JST Macrohistory + Gold Prices ===
```

```

print('Loading JST Macrohistory Database...')
jst = pd.read_excel('/home/user/JSTdatasetR6.xlsx')
print(f' Raw shape: {jst.shape}')

jst = jst[['year', 'iso', 'eq_tr', 'bill_rate', 'cpi', 'xrusd']].copy()
jst = jst.dropna(subset=['year', 'iso'])
jst['year'] = jst['year'].astype(int)

# Inflation from CPI levels
jst['inflation'] = jst.groupby('iso')['cpi'].pct_change()

# Real returns
jst['eq_real'] = (1 + jst['eq_tr']) / (1 + jst['inflation']) - 1
jst['bill_real'] = (1 + jst['bill_rate']) / (1 + jst['inflation']) - 1

# --- Gold: USD prices converted to local currency via exchange rate ---
print('\nDownloading gold price data...')
gold_url = 'https://raw.githubusercontent.com/datasets/gold-prices/main/data/monthly.csv'
gold_monthly_df = pd.read_csv(gold_url)
gold_monthly_df['Date'] = pd.to_datetime(gold_monthly_df['Date'])
gold_monthly_df['year'] = gold_monthly_df['Date'].dt.year

gold_annual = gold_monthly_df.sort_values('Date').groupby('year')['Price'].last().reset_index()
gold_annual.columns = ['year', 'gold_usd']
print(f' Gold USD prices: {gold_annual["year"].min()} to {gold_annual["year"].max()}')

# Merge gold USD price with JST
jst = jst.merge(gold_annual[['year', 'gold_usd']], on='year', how='left')

# Local-currency gold price = USD gold price × exchange rate to USD
# This correctly captures gold's value during crises:
# - When Germany hyperinflated, xrusd skyrocketed → local gold price skyrocketed
# - When Japan's yen collapsed post-WWII, gold in yen terms soared
# - For the US, xrusd ≈ 1, so local gold price = USD gold price
jst['gold_local'] = jst['gold_usd'] * jst['xrusd']
jst['gold_local_ret'] = jst.groupby('iso')['gold_local'].pct_change()

# Gold real return = local-currency gold return deflated by local CPI
jst['gold_real'] = (1 + jst['gold_local_ret']) / (1 + jst['inflation']) - 1

jst_clean = jst.dropna(subset=['eq_real', 'bill_real', 'inflation']).copy()

countries = sorted(jst_clean['iso'].unique())
print(f'\n=== JST Data Summary ===')
print(f'Countries ({len(countries)}): {", ".join(countries)}')
for iso in countries:
    sub = jst_clean[jst_clean['iso'] == iso]
    yr_min, yr_max = sub['year'].min(), sub['year'].max()
    n = len(sub)
    gold_n = sub['gold_real'].notna().sum()
    print(f' {iso}: {yr_min}-{yr_max} ({n} years, {gold_n} with gold)')

# Sanity checks
print('\n=== Sanity Checks ===')
for iso in ['USA', 'DEU', 'JPN', 'GBR']:
    sub = jst_clean[jst_clean['iso'] == iso]
    if len(sub) > 0:
        gold_sub = sub['gold_real'].dropna()
        print(f'{iso}: eq_real mean={sub["eq_real"].mean():.3f}, bill_real mean={sub["bill_real"].mean():.3f}, gold_real mean={gold_sub.mean():.3f}, inflation mean={sub["inflation"].mean():.3f}')

# Show Germany hyperinflation with local gold prices
deu = jst_clean[jst_clean['iso'] == 'DEU']
hyper = deu[deu['year'].isin([1919, 1920, 1921, 1922, 1923, 1924])]
if len(hyper) > 0:
    print(f'\nGermany crisis years (gold via exchange rate):')
    display_cols = ['year', 'inflation', 'eq_real', 'bill_real', 'gold_real']
    available = [c for c in display_cols if c in hyper.columns]
    print(hyper[available].to_string(index=False))

```

```
# Show Japan post-WWII
jpn = jst_clean[jst_clean['iso'] == 'JPN']
jpn_crisis = jpn[jpn['year'].isin([1944, 1945, 1946, 1947, 1948, 1949])]
if len(jpn_crisis) > 0:
    print(f'\nJapan crisis years (gold via exchange rate):')
    available = [c for c in display_cols if c in jpn_crisis.columns]
    print(jpn_crisis[available].to_string(index=False))
```

Loading JST Macrohistory Database...
Raw shape: (2718, 59)

Downloading gold price data...
Gold USD prices: 1833 to 2025

=== JST Data Summary ===

Countries (16): AUS, BEL, CHE, DEU, DNK, ESP, FIN, FRA, GBR, ITA, JPN, NLD, NOR, PRT, SWE, USA

AUS: 1871-2020 (147 years, 147 with gold)
BEL: 1871-2020 (146 years, 146 with gold)
CHE: 1900-2020 (121 years, 121 with gold)
DEU: 1871-2020 (144 years, 144 with gold)
DNK: 1875-2020 (146 years, 146 with gold)
ESP: 1900-2020 (117 years, 117 with gold)
FIN: 1896-2020 (125 years, 125 with gold)
FRA: 1871-2020 (143 years, 143 with gold)
GBR: 1871-2020 (150 years, 150 with gold)
ITA: 1871-2020 (130 years, 130 with gold)
JPN: 1886-2020 (133 years, 133 with gold)
NLD: 1900-2020 (121 years, 121 with gold)
NOR: 1881-2020 (140 years, 140 with gold)
PRT: 1880-2020 (141 years, 141 with gold)
SWE: 1871-2020 (150 years, 150 with gold)
USA: 1872-2020 (149 years, 149 with gold)

=== Sanity Checks ===

USA: eq_real mean=0.084, bill_real mean=0.021, gold_real mean=0.021, inflation mean=0.021
DEU: eq_real mean=0.087, bill_real mean=-0.003, gold_real mean=0.028, inflation mean=0.160
JPN: eq_real mean=0.088, bill_real mean=-0.000, gold_real mean=0.011, inflation mean=0.115
GBR: eq_real mean=0.068, bill_real mean=0.011, gold_real mean=0.020, inflation mean=0.032

Germany crisis years (gold via exchange rate):

year	inflation	eq_real	bill_real	gold_real
1919	0.328194	0.197187	-0.212238	1.600572
1920	1.454395	-0.313465	-0.577941	0.347815
1921	0.310811	0.707487	-0.210489	0.258949
1922	10.237113	0.930606	-0.905723	0.611069
1924	7.291767	-0.874395	-0.866000	-0.080775

Japan crisis years (gold via exchange rate):

year	inflation	eq_real	bill_real	gold_real
1944	0.352921	-0.207483	-0.241345	-0.260858
1945	9.756406	-0.903472	-0.904578	-0.846672
1948	0.759274	-0.161530	-0.412485	-0.259626
1949	0.318874	0.615534	-0.215088	0.328678

International Portfolio Definitions

The Five-Factor and Kelly portfolios are dropped from Part 2 — there are no factor loadings in the JST data, and annual frequency is too coarse for monthly Kelly sizing. The risky sleeve is simplified to 2x leveraged equity (no treasuries or crash insurance internationally). A **gold-barbell** variant replaces T-bills with gold in the safe allocation.

Strategy	Formula	Rationale
60/40	0.60 × equity + 0.40 × bills	Traditional balanced
80/20	0.80 × equity + 0.20 × bills	Aggressive traditional

Strategy	Formula	Rationale
Barbell (bills)	$0.85 \times \text{bills} + 0.15 \times \max(2 \times \text{equity} - \text{bills}, -1)$	Bills safe, 2x leveraged equity risky
Barbell (gold)	$0.85 \times \text{gold} + 0.15 \times \max(2 \times \text{equity} - \text{bills}, -1)$	Gold safe, same risky sleeve

The risky sleeve is capped at -100% (the leveraged position can lose everything but cannot go further negative — no margin calls in this model). Pre-1971 gold real return $\approx$ -inflation (gold standard fixed price). Post-1971: market price returns.

```
In [13]: # === Construct International Portfolios ===

intl_strategies = ['60/40', '80/20', 'Barbell (bills)', 'Barbell (gold)']

def compute_intl_portfolios(row):
    eq = row['eq_real']
    bill = row['bill_real']
    gold = row['gold_real']

    risky = max(2 * eq - bill, -1.0)

    p_6040 = 0.60 * eq + 0.40 * bill
    p_8020 = 0.80 * eq + 0.20 * bill
    p_barbell_bills = 0.85 * bill + 0.15 * risky
    p_barbell_gold = 0.85 * gold + 0.15 * risky if pd.notna(gold) else np.nan

    return pd.Series({
        '60/40': p_6040,
        '80/20': p_8020,
        'Barbell (bills)': p_barbell_bills,
        'Barbell (gold)': p_barbell_gold,
    })

intl_ports = jst_clean.join(jst_clean.apply(compute_intl_portfolios, axis=1))

country_results = {}
for iso in countries:
    sub = intl_ports[intl_ports['iso'] == iso].set_index('year')
    wealth = {}
    cagr = {}
    for strat in intl_strategies:
        s = sub[strat].dropna()
        if len(s) > 5:
            cum = (1 + s).cumprod()
            wealth[strat] = cum
            n_yrs = len(s)
            cagr[strat] = cum.iloc[-1] ** (1 / n_yrs) - 1
    if wealth:
        country_results[iso] = {
            'wealth': wealth,
            'cagr': cagr,
            'returns': sub[intl_strategies],
            'n_years': len(sub),
            'year_range': (sub.index.min(), sub.index.max()),
        }

print(f'=== International Portfolio Results ===')
print(f'{"Country":>8} {"Years":>12} {"N":>4}', end='')
for s in intl_strategies:
    print(f'{"s":>18}', end='')
print()
print('-' * (28 + 18 * len(intl_strategies)))
for iso in sorted(country_results.keys()):
    cr = country_results[iso]
    yr = cr['year_range']
    print(f'{"iso":>8} {"yr[0]":>4} {"yr[1]":>4} {"cr["n_years"]:>4}', end='')
    for s in intl_strategies:
        if s in cr['cagr']:
            print(f'{"s":>18} {"cr["cagr"]:>10}', end='')
        else:
            print(f'{"s":>18} {"cr["wealth"]:>10}', end='')
    print()
```

```

        print(f'{cr["cagr"][s]:>17.2%}', end='')
    else:
        print(f'{"N/A":>18%}', end='')
    print()

print(f'\nCountries with results: {len(country_results)}')

```

=== International Portfolio Results ===

Country	Years	N	60/40	80/20	Barbell (bills)	Barbell (gold)
AUS	1871-2020	147	5.29%	6.25%	3.67%	3.19%
BEL	1871-2020	146	3.64%	4.15%	2.56%	3.14%
CHE	1900-2020	121	3.56%	4.30%	2.22%	2.00%
DEU	1871-2020	144	3.12%	4.19%	1.02%	3.52%
DNK	1875-2020	146	5.30%	5.95%	4.13%	2.69%
ESP	1900-2020	117	2.60%	3.36%	1.21%	2.21%
FIN	1896-2020	125	3.06%	4.13%	1.03%	3.35%
FRA	1871-2020	143	1.24%	1.63%	0.38%	2.24%
GBR	1871-2020	150	3.90%	4.67%	2.57%	2.81%
ITA	1871-2020	130	2.44%	2.87%	1.34%	2.53%
JPN	1886-2020	133	2.91%	3.95%	0.98%	1.13%
NLD	1900-2020	121	3.78%	4.52%	2.36%	2.60%
NOR	1881-2020	140	3.05%	3.57%	2.01%	2.33%
PRT	1880-2020	141	0.75%	0.75%	0.36%	1.74%
SWE	1871-2020	150	4.80%	5.64%	3.30%	2.98%
USA	1872-2020	149	5.26%	6.11%	3.76%	3.13%

Countries with results: 16

In [14]: # === KEY CHART 1: Terminal Wealth by Country and Strategy ===

```

intl_colors = {
    '60/40': COLORS['60/40'],
    '80/20': COLORS['80/20'],
    'Barbell (bills)': COLORS['Barbell'],
    'Barbell (gold)': COLORS['Barbell (gold)'],
}

scatter_data = []
for iso in sorted(country_results.keys()):
    cr = country_results[iso]
    for strat in intl_strategies:
        if strat in cr['wealth']:
            tw_val = cr['wealth'][strat].iloc[-1]
            scatter_data.append({'iso': iso, 'strategy': strat, 'terminal_wealth': tw_val})

scatter_df = pd.DataFrame(scatter_data)

fig, ax = plt.subplots(figsize=(16, 8))
sorted_countries = sorted(country_results.keys())
x_positions = {iso: i for i, iso in enumerate(sorted_countries)}
offsets = {s: (i - 1.5) * 0.18 for i, s in enumerate(intl_strategies)}

# Alternating background bands for country separation
for i, iso in enumerate(sorted_countries):
    if i % 2 == 0:
        ax.axvspan(i - 0.5, i + 0.5, alpha=0.04, color='grey', zorder=0)

for strat in intl_strategies:
    strat_data = scatter_df[scatter_df['strategy'] == strat]
    x = [x_positions[iso] + offsets[strat] for iso in strat_data['iso']]
    y = strat_data['terminal_wealth'].values
    ax.scatter(x, y, color=intl_colors[strat], s=90, alpha=0.85,
               label=strat, edgecolors='white', linewidth=0.5, zorder=3)

ax.set_yscale('log')
ax.set_xticks(list(x_positions.values()))
ax.set_xticklabels(list(x_positions.keys()), fontsize=10)

```

```

ax.axhline(y=1, color='black', ls=':', lw=1, alpha=0.5, label='$1 (breakeven)')
ax.set_ylabel('Terminal Real Wealth (log scale, $1 invested)', fontsize=12)
ax.set_title('Terminal Wealth by Country and Strategy (1870-2020, real terms)', fontsize=14)
ax.legend(fontsize=10, loc='upper left')
ax.grid(True, alpha=0.3, axis='y')

# Add vertical separators between countries
for i in range(len(sorted_countries) - 1):
    ax.axvline(x=i + 0.5, color='grey', lw=0.3, alpha=0.3, zorder=0)

plt.tight_layout()
plt.show()

print('=== Terminal Wealth Extremes ===')
for strat in intl_strategies:
    strat_data = scatter_df[scatter_df['strategy'] == strat]
    best = strat_data.loc[strat_data['terminal_wealth'].idxmax()]
    worst = strat_data.loc[strat_data['terminal_wealth'].idxmin()]
    print(f'{strat:>20}: best={best["iso"]} ${{best["terminal_wealth"]:>12,.0f}}, '
          f'worst={worst["iso"]} ${{worst["terminal_wealth"]:>12,.2f}}')

```



```

=== Terminal Wealth Extremes ===
           60/40: best=USA $      2,089, worst=PRT $      2.88
           80/20: best=AUS $      7,442, worst=PRT $      2.87
Barbell (bills): best=DNK $       368, worst=PRT $      1.65
Barbell (gold): best=DEU $       146, worst=JPN $      4.47

```

Reading the Scatter — and What It Hides

The terminal wealth scatter shows 80/20 producing the highest terminal wealth in most countries. But full-sample terminal wealth hides important structure.

Gold never beats equities over the full sample — except Portugal, where it is essentially a tie in the weakest equity market. Over 150 years, equities compound faster than gold in every stable country.

But gold beats equities for decades at a time in crisis-prone countries. A sub-period analysis testing every 20+ year window in each country reveals thousands of multi-decade stretches where gold outperformed:

Country	Windows where gold beat equities (20+ yr)	Longest window
France	6,302	146 yr (1873–2018)
Portugal	4,781	148 yr (nearly the entire sample)
Italy	2,071	111 yr (1872–1982)

Country	Windows where gold beat equities (20+ yr)	Longest window
Belgium	624	83 yr (1898–1980)
Spain	577	77 yr (1904–1980)
Germany	421	67 yr (1880–1948)
Norway	420	74 yr
Finland	278	63 yr
Japan	205	43 yr (1970–2012)
Sweden	175	35 yr
Netherlands	138	40 yr
USA	108	29 yr (1959–1987)
UK	59	46 yr (1929–1974)
Australia	70	28 yr
Switzerland	50	27 yr
Denmark	52	34 yr

These are end-to-end measurements: an investor who bought gold at the start year and equities at the same start year would have more wealth in gold at the end year. The chart below shows what that experience looked like for an 80/20 vs gold-barbell investor during each country's longest such window.

```
In [15]: # === Longest End-to-End Windows Where Gold Beat Equities – Portfolio Impact ===
# For each country, find the longest window [start, end] where cumulative gold > cumulative
# Then plot 80/20 vs gold-barbell over that window.

streak_data_endtoend = {}
for iso in sorted(country_results.keys()):
    sub = jst_clean[jst_clean['iso'] == iso].sort_values('year').reset_index(drop=True)
    eq = sub['eq_real'].values
    gold = sub['gold_real'].values
    years = sub['year'].values
    n = len(eq)

    best_start, best_end, best_len = 0, 0, 0
    for i in range(n):
        cum_eq = 1.0
        cum_gold = 1.0
        for j in range(i, n):
            if np.isnan(eq[j]) or np.isnan(gold[j]):
                break
            cum_eq *= (1 + eq[j])
            cum_gold *= (1 + gold[j])
            if cum_gold >= cum_eq and (j - i + 1) >= 20:
                if (j - i + 1) > best_len:
                    best_len = j - i + 1
                    best_start = i
                    best_end = j

    if best_len >= 20:
        streak_data_endtoend[iso] = {
            'start_year': int(years[best_start]),
            'end_year': int(years[best_end]),
            'length': best_len,
        }

sorted_streaks_e2e = sorted(streak_data_endtoend.items(), key=lambda x: x[1]['length'], reverse=True)
top_n = min(12, len(sorted_streaks_e2e))
plot_streaks_e2e = sorted_streaks_e2e[:top_n]
```

```

country_names = {
    'AUS': 'Australia', 'BEL': 'Belgium', 'CHE': 'Switzerland', 'DEU': 'Germany',
    'DNK': 'Denmark', 'ESP': 'Spain', 'FIN': 'Finland', 'FRA': 'France',
    'GBR': 'United Kingdom', 'ITA': 'Italy', 'JPN': 'Japan', 'NLD': 'Netherlands',
    'NOR': 'Norway', 'PRT': 'Portugal', 'SWE': 'Sweden', 'USA': 'United States',
}

ncols = 4
nrows = (top_n + ncols - 1) // ncols
fig, axes = plt.subplots(nrows, ncols, figsize=(20, 4.5 * nrows))
axes_flat = axes.flatten()

for idx, (iso, info) in enumerate(plot_streaks_e2e):
    ax = axes_flat[idx]
    sub = jst_clean[jst_clean['iso'] == iso].sort_values('year')
    window = sub[(sub['year'] >= info['start_year']) & (sub['year'] <= info['end_year'])].values
    yrs = window['year'].values

    r_8020 = window['eq_real'].values * 0.80 + window['bill_real'].values * 0.20
    w_8020 = np.cumprod(1 + r_8020)

    risky = np.maximum(2 * window['eq_real'].values - window['bill_real'].values, -1.0)
    r_gbb = 0.85 * window['gold_real'].values + 0.15 * risky
    w_gbb = np.cumprod(1 + r_gbb)

    ax.fill_between(yrs, w_8020, w_gbb,
                    where=w_gbb >= w_8020, alpha=0.12, color=intl_colors['Barbell (gold)'],
                    interpolate=True)
    ax.fill_between(yrs, w_8020, w_gbb,
                    where=w_gbb < w_8020, alpha=0.12, color=intl_colors['80/20'],
                    interpolate=True)

    ax.plot(yrs, w_8020, color=intl_colors['80/20'], lw=2.2, label='80/20')
    ax.plot(yrs, w_gbb, color=intl_colors['Barbell (gold)'], lw=2.2, label='Barbell (gold)')
    ax.axhline(y=1, color='black', ls=':', lw=0.8, alpha=0.4)

    name = country_names.get(iso, iso)
    ax.set_title(f'{name} ({info["start_year"]}\u2013{info["end_year"]}, {info["length"]})',
                 fontsize=11, fontweight='bold')
    ax.set_yscale('symlog', lincthresh=1)
    ax.set_ylabel('Real Wealth ($1)', fontsize=9)
    ax.grid(True, alpha=0.2)

    f8 = w_8020[-1]
    fg = w_gbb[-1]
    ax.annotate(f'${fg:.1f}', xy=(yrs[-1], fg),
                fontsize=8, color=intl_colors['Barbell (gold)'], fontweight='bold',
                ha='right', va='bottom')
    ax.annotate(f'${f8:.1f}', xy=(yrs[-1], f8),
                fontsize=8, color=intl_colors['80/20'], fontweight='bold',
                ha='right', va='top')

    if idx == 0:
        ax.legend(fontsize=8, loc='upper left')

for idx in range(top_n, len(axes_flat)):
    axes_flat[idx].set_visible(False)

fig.suptitle('Longest End-to-End Windows Where Gold Beat Equities\n'
             '$1 invested at the start of each window in 80/20 vs gold-barbell',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print(f'\n{"Country":>14} {"Window":>14} {"Length":>6} {"80/20 $1\u20192":>10} {"Gold-BB $1'
print('-' * 68)
for iso, info in plot_streaks_e2e:
    sub = jst_clean[jst_clean['iso'] == iso].sort_values('year')
    window = sub[(sub['year'] >= info['start_year']) & (sub['year'] <= info['end_year'])]

```

```

r_8020 = window['eq_real'].values * 0.80 + window['bill_real'].values * 0.20
w_8020 = np.cumprod(1 + r_8020)[-1]
risky = np.maximum(2 * window['eq_real'].values - window['bill_real'].values, -1.0)
w_gbb = np.cumprod(1 + 0.85 * window['gold_real'].values + 0.15 * risky)[-1]
name = country_names.get(iso, iso)
print(f'{name:>14} {info["start_year"]}-{info["end_year"]:>4} {info["length"]:>4}yr '
      f'${w_8020:>8,.1f} ${w_gbb:>10,.1f} {w_gbb/w_8020:>6.1f}x')

```

Longest End-to-End Windows Where Gold Beat Equities
\$1 invested at the start of each window in 80/20 vs gold-barbell



Country	Window Length	80/20 \$1→	Gold-BB \$1→	Ratio
Portugal 1880-2020	141yr	\$ 2.9	\$ 11.4	4.0x
France 1882-2020	132yr	\$ 3.2	\$ 14.7	4.7x
Italy 1925-2012	88yr	\$ 2.6	\$ 11.5	4.4x
Belgium 1899-1980	78yr	\$ 4.8	\$ 10.9	2.3x
Norway 1909-1982	74yr	\$ 1.8	\$ 3.4	1.9x
Germany 1872-1941	69yr	\$ 1.5	\$ 6.0	3.9x
Spain 1918-1983	63yr	\$ 2.0	\$ 6.8	3.5x
Finland 1918-1980	63yr	\$ 3.5	\$ 23.5	6.8x
United Kingdom 1929-1974	46yr	\$ 2.5	\$ 4.9	2.0x
Japan 1970-2012	43yr	\$ 2.8	\$ 4.6	1.7x
Netherlands 1907-1946	40yr	\$ 1.9	\$ 2.8	1.4x
Sweden 1898-1932	35yr	\$ 1.0	\$ 0.8	0.8x

The end-to-end windows above include periods where equity temporarily pulled ahead before gold regained the lead. A stricter test: when did gold pull ahead and **stay** ahead, year after year, without interruption?

The chart below finds, for each country, the longest consecutive run where an investor starting at year zero would have been better off in gold than equities at every single checkpoint. These are the periods of unbroken pain for the equity-heavy investor — years where switching to gold would have looked right not just in hindsight, but in real time, every year.

```

In [16]: # === Longest Continuous Gold-Ahead Streaks – Portfolio Impact ===
# The end-to-end windows include periods where equity temporarily pulled ahead
# before gold regained the lead. A stricter test: when did gold pull ahead and
# STAY ahead, year after year, without interruption?

streak_data_continuous = {}

```

```

for iso in sorted(country_results.keys()):
    sub = jst_clean[jst_clean['iso'] == iso].sort_values('year').reset_index(drop=True)
    eq = sub['eq_real'].values
    gold = sub['gold_real'].values
    years = sub['year'].values
    n = len(eq)

    best_start, best_len = 0, 0
    for i in range(n):
        cum_eq = 1.0
        cum_gold = 1.0
        streak = 0
        for j in range(i, n):
            if np.isnan(eq[j]) or np.isnan(gold[j]):
                break
            cum_eq *= (1 + eq[j])
            cum_gold *= (1 + gold[j])
            if cum_gold > cum_eq:
                streak += 1
            else:
                break
        if streak > best_len:
            best_len = streak
            best_start = i

    if best_len >= 10:
        streak_data_continuous[iso] = {
            'start_year': int(years[best_start]),
            'end_year': int(years[best_start + best_len - 1]),
            'length': best_len,
        }

sorted_streaks_cont = sorted(streak_data_continuous.items(), key=lambda x: x[1]['length'],
top_n = min(16, len(sorted_streaks_cont))
plot_streaks_cont = sorted_streaks_cont[:top_n]

ncols = 4
nrows = (top_n + ncols - 1) // ncols
fig, axes = plt.subplots(nrows, ncols, figsize=(20, 4.5 * nrows))
axes_flat = axes.flatten()

for idx, (iso, info) in enumerate(plot_streaks_cont):
    ax = axes_flat[idx]
    sub = jst_clean[jst_clean['iso'] == iso].sort_values('year')
    window = sub[(sub['year'] >= info['start_year']) & (sub['year'] <= info['end_year'])].values
    yrs = window['year'].values

    r_8020 = window['eq_real'].values * 0.80 + window['bill_real'].values * 0.20
    w_8020 = np.cumprod(1 + r_8020)

    risky = np.maximum(2 * window['eq_real'].values - window['bill_real'].values, -1.0)
    r_gbb = 0.85 * window['gold_real'].values + 0.15 * risky
    w_gbb = np.cumprod(1 + r_gbb)

    ax.fill_between(yrs, w_8020, w_gbb,
                    where=w_gbb >= w_8020, alpha=0.12, color=intl_colors['Barbell (gold)'],
                    interpolate=True)
    ax.fill_between(yrs, w_8020, w_gbb,
                    where=w_gbb < w_8020, alpha=0.12, color=intl_colors['80/20'],
                    interpolate=True)

    ax.plot(yrs, w_8020, color=intl_colors['80/20'], lw=2.2, label='80/20')
    ax.plot(yrs, w_gbb, color=intl_colors['Barbell (gold)'], lw=2.2, label='Barbell (gold)')
    ax.axhline(y=1, color='black', ls=':', lw=0.8, alpha=0.4)

    name = country_names.get(iso, iso)
    ax.set_title(f'{name} ({info["start_year"]}\u2013{info["end_year"]}, {info["length"]})',
                fontsize=11, fontweight='bold')
    ax.set_yscale('symlog', linthresh=1)

```

```

ax.set_ylabel('Real Wealth ($1)', fontsize=9)
ax.grid(True, alpha=0.2)

f8 = w_8020[-1]
fg = w_gbb[-1]
ax.annotate(f'${fg:.1f}', xy=(yrs[-1], fg),
            fontsize=8, color=intl_colors['Barbell (gold)'], fontweight='bold',
            ha='right', va='bottom')
ax.annotate(f'${f8:.1f}', xy=(yrs[-1], f8),
            fontsize=8, color=intl_colors['80/20'], fontweight='bold',
            ha='right', va='top')

if idx == 0:
    ax.legend(fontsize=8, loc='upper left')

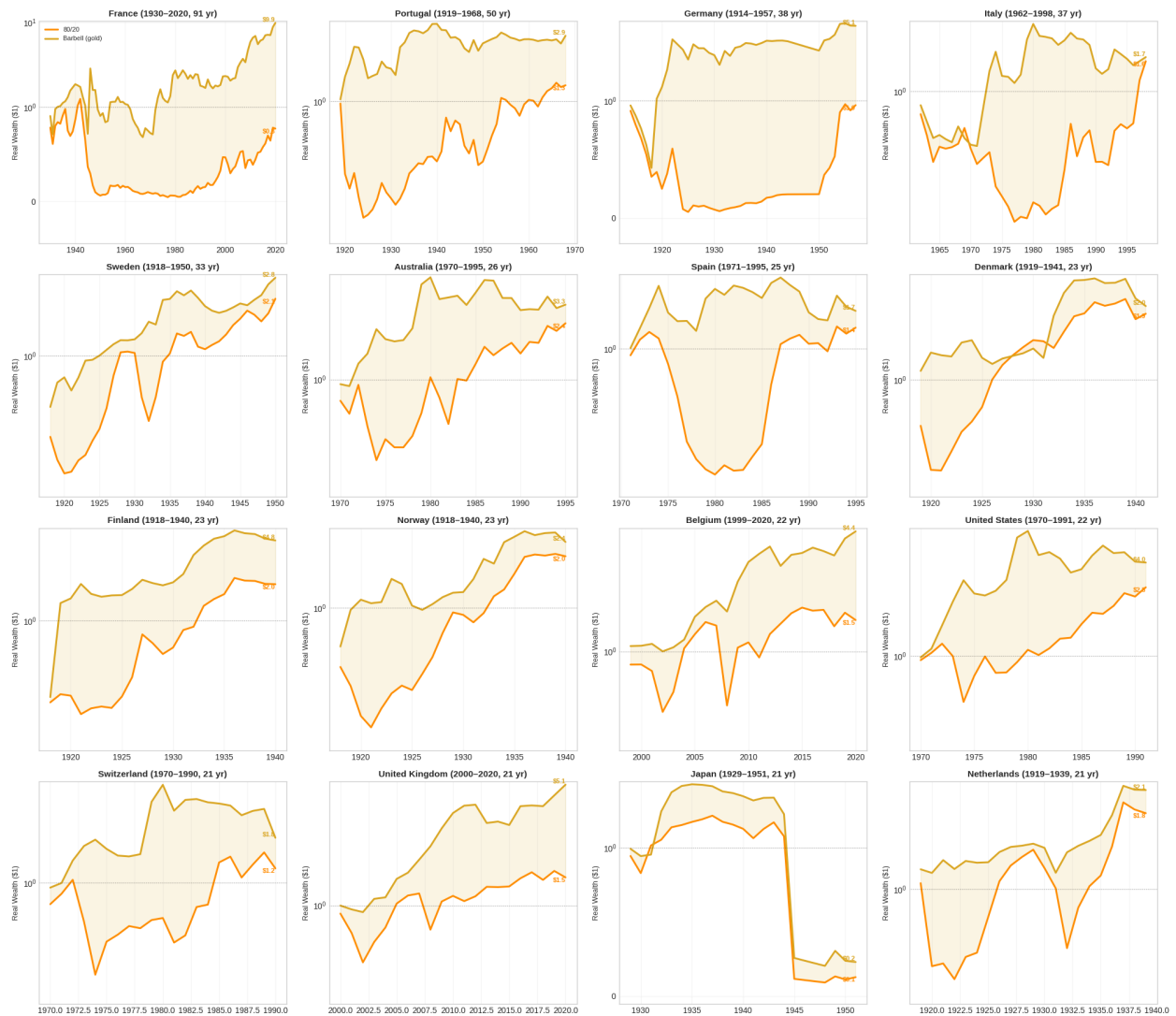
for idx in range(top_n, len(axes_flat)):
    axes_flat[idx].set_visible(False)

fig.suptitle('Longest Continuous Periods Where Gold Led Equities\n'
            'Each panel: $1 invested at the start in 80/20 vs gold-barbell – gold ahead ev
            fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print(f'\n{"Country":>14} {"Window":>14} {"Streak":>6} {"80/20 $1\u2192":>10} {"Gold-BB $1'
print('-' * 68)
for iso, info in plot_streaks_cont:
    sub = jst_clean[jst_clean['iso'] == iso].sort_values('year')
    window = sub[(sub['year'] >= info['start_year']) & (sub['year'] <= info['end_year'])]
    r_8020 = window['eq_real'].values * 0.80 + window['bill_real'].values * 0.20
    w_8020 = np.cumprod(1 + r_8020)[-1]
    risky = np.maximum(2 * window['eq_real'].values - window['bill_real'].values, -1.0)
    w_gbb = np.cumprod(1 + 0.85 * window['gold_real'].values + 0.15 * risky)[-1]
    name = country_names.get(iso, iso)
    print(f'{name:>14} {info["start_year"]}-{info["end_year"]:>4} {info["length"]:>4}yr '
          f'${w_8020:>8,.1f} ${w_gbb:>10,.1f} {w_gbb/w_8020:>6.1f}x')

```

Longest Continuous Periods Where Gold Led Equities
Each panel: \$1 invested at the start in 80/20 vs gold-barbell — gold ahead every year



Country	Window Streak	80/20 \$1→	Gold-BB \$1→	Ratio
France 1930–2020	91yr	\$0.8	\$9.9	12.8x
Portugal 1919–1968	50yr	\$1.3	\$2.9	2.2x
Germany 1914–1957	38yr	\$1.0	\$5.1	5.3x
Italy 1962–1998	37yr	\$1.6	\$1.7	1.1x
Sweden 1918–1950	33yr	\$2.1	\$2.8	1.3x
Australia 1970–1995	26yr	\$2.4	\$3.3	1.3x
Spain 1971–1995	25yr	\$1.4	\$1.7	1.3x
Denmark 1919–1941	23yr	\$1.9	\$2.0	1.1x
Finland 1918–1940	23yr	\$2.0	\$4.8	2.4x
Norway 1918–1940	23yr	\$2.0	\$2.4	1.2x
Belgium 1999–2020	22yr	\$1.5	\$4.4	3.0x
United States 1970–1991	22yr	\$2.8	\$4.0	1.4x
Switzerland 1970–1990	21yr	\$1.2	\$1.8	1.5x
United Kingdom 2000–2020	21yr	\$1.5	\$5.1	3.5x
Japan 1929–1951	21yr	\$0.1	\$0.2	1.8x
Netherlands 1919–1939	21yr	\$1.8	\$2.1	1.2x

In [17]: # === CAGR Heatmap: Countries x Strategies ===

```

cagr_matrix = {}
for iso in sorted(country_results.keys()):
    cr = country_results[iso]
    cagr_matrix[iso] = {s: cr['cagr'].get(s, np.nan) for s in intl_strategies}

cagr_df = pd.DataFrame(cagr_matrix).T * 100
cagr_df = cagr_df.sort_values('80/20', ascending=False)

fig, ax = plt.subplots(figsize=(10, 10))

```

```

vals = cagr_df.values[~np.isnan(cagr_df.values)]
if vals.min() < 0 < vals.max():
    norm = TwoSlopeNorm(vmin=vals.min(), vcenter=0, vmax=vals.max())
else:
    norm = None # All values same sign, use default normalization
im = ax.imshow(cagr_df.values, cmap='RdYlGn', norm=norm, aspect='auto')

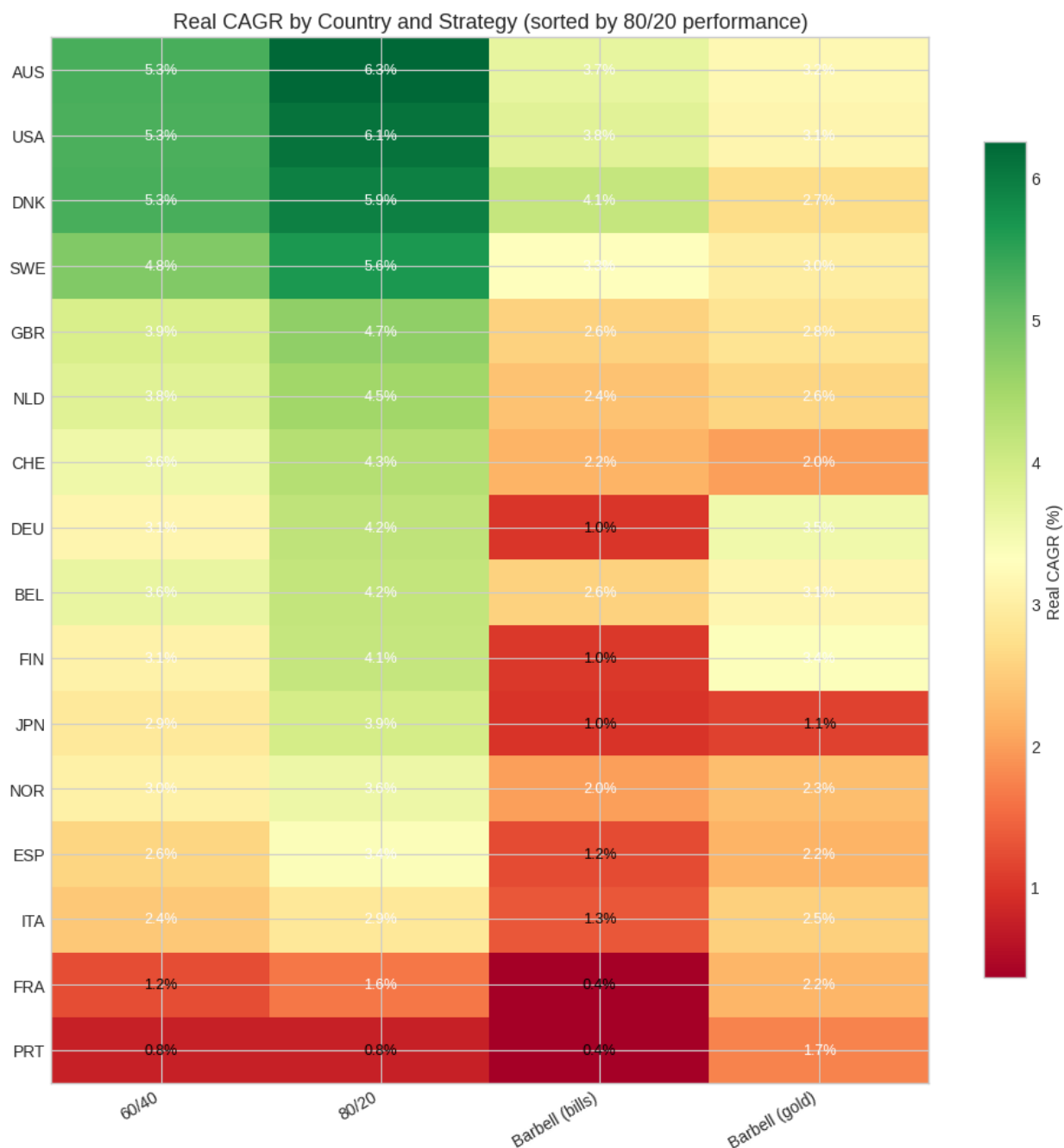
ax.set_xticks(range(len(intl_strategies)))
ax.set_xticklabels(intl_strategies, fontsize=10, rotation=30, ha='right')
ax.set_yticks(range(len(cagr_df)))
ax.set_yticklabels(cagr_df.index, fontsize=10)

for i in range(len(cagr_df)):
    for j in range(len(intl_strategies)):
        val = cagr_df.iloc[i, j]
        if not np.isnan(val):
            color = 'white' if abs(val) > vals.std() else 'black'
            ax.text(j, i, f'{val:.1f}%', ha='center', va='center', fontsize=9, color=color)

ax.set_title('Real CAGR by Country and Strategy (sorted by 80/20 performance)', fontsize=12)
fig.colorbar(im, ax=ax, label='Real CAGR (%)', shrink=0.8)
plt.tight_layout()
plt.show()

print('=== CAGR Summary ===')
print(f'Top 80/20 countries: {list(cagr_df.head(5).index)}')
print(f'Bottom 80/20 countries: {list(cagr_df.tail(5).index)}')
print(f'\nCountries where gold-barbell beats 80/20:')
for iso in cagr_df.index:
    bb_gold = cagr_df.loc[iso, 'Barbell (gold)']
    p80 = cagr_df.loc[iso, '80/20']
    if pd.notna(bb_gold) and pd.notna(p80) and bb_gold > p80:
        print(f' {iso}: gold-barbell {bb_gold:.1f}% vs 80/20 {p80:.1f}%')

```



=== CAGR Summary ===

Top 80/20 countries: ['AUS', 'USA', 'DNK', 'SWE', 'GBR']

Bottom 80/20 countries: ['NOR', 'ESP', 'ITA', 'FRA', 'PRT']

Countries where gold-barbell beats 80/20:

FRA: gold-barbell 2.2% vs 80/20 1.6%

PRT: gold-barbell 1.7% vs 80/20 0.8%

```
In [18]: # === Regime Failure Analysis ===

regime_failures_data = jst_clean[jst_clean['bill_real'] < -0.20].copy()

# Compute portfolio returns for these rows
regime_port_rets = regime_failures_data.apply(compute_intl_portfolios, axis=1)
regime_failures_data = regime_failures_data.join(regime_port_rets)

print('=== Regime Failure Events: Years Where Bill Real Return < -20% ===')
events_by_country = regime_failures_data.groupby('iso').size().sort_values(ascending=False)
print(f'\nEvents per country:')
for iso, count in events_by_country.items():
    years = sorted(regime_failures_data[regime_failures_data['iso'] == iso]['year'].tolist())
    print(f' {iso}: {count} events \u2014 {years}')
```

```

print(f'\nTotal events: {len(regime_failures_data)}')

print(f'\n=== Mean Real Return During Regime Failure Events ===')
for strat in intl_strategies:
    vals = regime_failures_data[strat].dropna()
    if len(vals) > 0:
        print(f' {strat:>20}: mean={vals.mean():>+7.1%}, median={vals.median():>+7.1%}, n=

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

ax = axes[0]
events_by_country.plot(kind='barh', ax=ax, color='firebrick', alpha=0.7)
ax.set_xlabel('Number of Years with Bill Real Return < -20%')
ax.set_title('Regime Failure Frequency by Country', fontsize=13)
ax.grid(True, alpha=0.3, axis='x')

ax = axes[1]
failure_means = {}
for strat in intl_strategies:
    vals = regime_failures_data[strat].dropna()
    failure_means[strat] = vals.mean() * 100
bars = ax.bar(range(len(intl_strategies)), list(failure_means.values()),
              color=[intl_colors[s] for s in intl_strategies], alpha=0.8)
ax.set_xticks(range(len(intl_strategies)))
ax.set_xticklabels(intl_strategies, fontsize=10, rotation=20, ha='right')
ax.set_ylabel('Mean Real Return (%)')
ax.set_title('Mean Strategy Return During Regime Failures', fontsize=13)
ax.axhline(y=0, color='black', ls='-', lw=0.8)
ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

```

=== Regime Failure Events: Years Where Bill Real Return < -20% ===

Events per country:

```

DEU: 7 events – [1916, 1917, 1919, 1920, 1921, 1922, 1924]
FIN: 7 events – [1916, 1917, 1918, 1945, 1946, 1947, 1948]
PRT: 6 events – [1918, 1920, 1921, 1923, 1924, 1974]
JPN: 6 events – [1918, 1919, 1944, 1945, 1948, 1949]
ITA: 4 events – [1943, 1944, 1945, 1947]
FRA: 4 events – [1945, 1946, 1947, 1948]
BEL: 3 events – [1940, 1941, 1942]
ESP: 1 events – [1918]
NOR: 1 events – [1918]
SWE: 1 events – [1918]

```

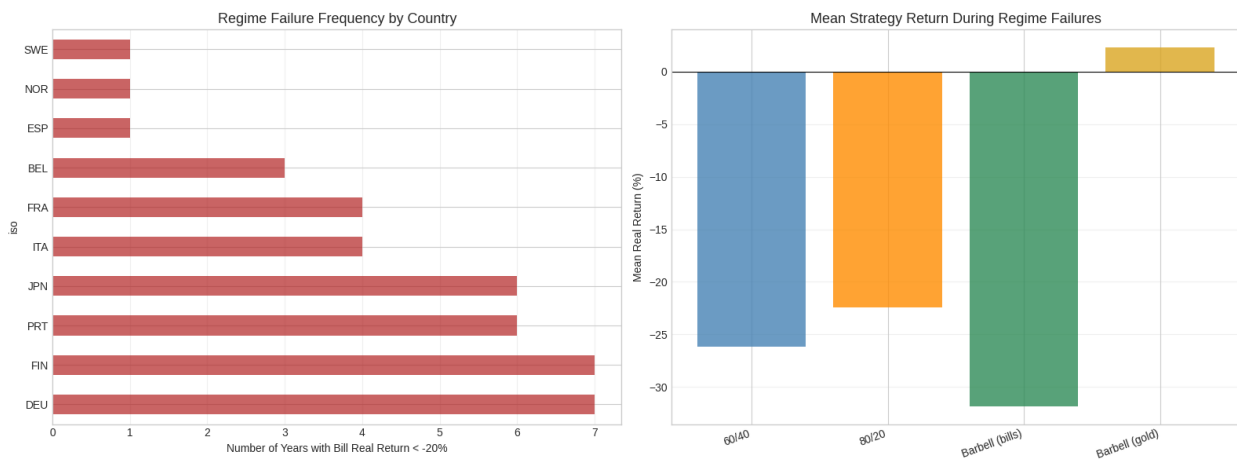
Total events: 40

=== Mean Real Return During Regime Failure Events ===

```

        60/40: mean= -26.2%, median= -29.0%, n=40
        80/20: mean= -22.4%, median= -26.7%, n=40
Barbell (bills): mean= -31.8%, median= -29.2%, n=40
Barbell (gold): mean=  +2.3%, median= -18.0%, n=40

```



The Bills Problem — and Gold's Answer

T-bills are government promises denominated in government currency. They are "safe" only as long as the currency retains purchasing power:

- **Germany 1920-1923:** Hyperinflation destroyed the real value of all mark-denominated assets. Bills lost >99% of purchasing power. But gold's mark-denominated price *skyrocketed* — the exchange rate captured the currency's collapse, and gold holders preserved wealth even as the monetary system disintegrated. (Note: 1923, the worst year, is absent from the JST dataset.)
- **Japan 1945-1949:** Post-war inflation and currency reform wiped out nominal savings. But the yen's collapse against the dollar meant gold in yen terms appreciated enormously — gold holders saw massive gains in 1946 even as bill holders lost everything.
- **Italy 1943-1947:** War and post-war inflation produced years of deeply negative real bill returns.
- **Finland 1916-1918 and 1945-1948:** Civil war, independence, and post-WWII reparations produced currency instability.

This is not historical curiosity. Argentina, Turkey, Venezuela, and Lebanon have experienced analogous episodes in living memory.

Gold works during these crises because it is priced in a global market. When a country debases its currency, the local price of gold rises to compensate — this is the exchange rate doing its job. The "safe" asset in the gold-barbell appreciates during exactly the scenario where the "safe" asset in the bills-barbell is destroyed.

```
In [19]: # === Drawdown Comparison Across Countries ===

max_dd_data = []
for iso in sorted(country_results.keys()):
    cr = country_results[iso]
    for strat in intl_strategies:
        if strat in cr['wealth']:
            w = cr['wealth'][strat]
            rm = w.cummax()
            dd = (w - rm) / rm
            max_dd = dd.min()
            max_dd_data.append({'iso': iso, 'strategy': strat, 'max_drawdown': max_dd})

max_dd_df = pd.DataFrame(max_dd_data)

fig, ax = plt.subplots(figsize=(12, 6))
bp_data = [max_dd_df[max_dd_df['strategy'] == s]['max_drawdown'].values * 100
            for s in intl_strategies]

bp = ax.boxplot(bp_data, labels=intl_strategies, patch_artist=True, widths=0.6,
                medianprops=dict(color='black', linewidth=2))
for patch, strat in zip(bp['boxes'], intl_strategies):
```

```

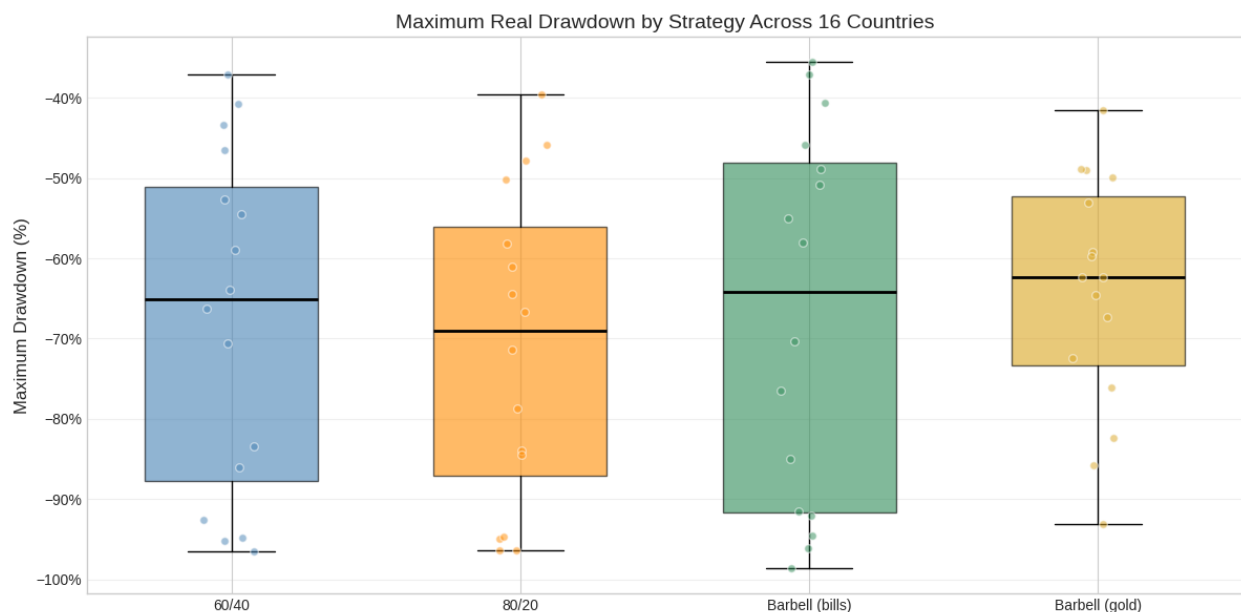
patch.set_facecolor(intl_colors[strat])
patch.set_alpha(0.6)

rng = np.random.RandomState(42)
for i, strat in enumerate(intl_strategies):
    strat_data = max_dd_df[max_dd_df['strategy'] == strat]
    jitter = rng.normal(0, 0.05, len(strat_data))
    ax.scatter(i + 1 + jitter, strat_data['max_drawdown'].values * 100,
               alpha=0.5, s=30, color=intl_colors[strat], edgecolors='white', zorder=3)

ax.set_ylabel('Maximum Drawdown (%)', fontsize=12)
ax.set_title('Maximum Real Drawdown by Strategy Across 16 Countries', fontsize=14)
ax.grid(True, alpha=0.3, axis='y')
ax.yaxis.set_major_formatter(PercentFormatter(decimals=0))
plt.tight_layout()
plt.show()

print('=== Max Drawdown Summary ===')
for strat in intl_strategies:
    vals = max_dd_df[max_dd_df['strategy'] == strat]['max_drawdown']
    print(f'{strat:>20}: median={vals.median():.1%}, worst={vals.min():.1%}, best={vals.max():.1%}')

```



```

=== Max Drawdown Summary ===
        60/40: median=-65.1%, worst=-96.6%, best=-37.1%
        80/20: median=-69.0%, worst=-96.4%, best=-39.6%
Barbell (bills): median=-64.2%, worst=-98.6%, best=-35.6%
Barbell (gold): median=-62.4%, worst=-93.1%, best=-41.5%

```

In [20]: *# === International Monte Carlo: Pooled Annual Returns ===*

```

np.random.seed(42)
n_sim = 10_000
n_years_mc = 30

pooled = {}
for strat in intl_strategies:
    all_ret = []
    for iso in country_results:
        cr = country_results[iso]
        if strat in cr['returns'].columns:
            vals = cr['returns'][strat].dropna().values
            all_ret.extend(vals.tolist())
    pooled[strat] = np.array(all_ret)

print('=== Pooled Return Summary ===')
for strat in intl_strategies:
    r = pooled[strat]
    print(f'{strat:>20}: n={len(r)}, mean={r.mean():.3f}, std={r.std():.3f}, ')

```

```

        f'min={r.min():.3f}, max={r.max():.3f}')

intl_tw = {}
for strat in intl_strategies:
    returns = pooled[strat]
    simulated = np.random.choice(returns, size=(n_sim, n_years_mc), replace=True)
    intl_tw[strat] = np.prod(1 + simulated, axis=1)

intl_tw_df = pd.DataFrame(intl_tw)

fig, ax = plt.subplots(figsize=(14, 6))
for strat in intl_strategies:
    log_tw = np.log10(np.clip(intl_tw_df[strat].values, 1e-6, None))
    ax.hist(log_tw, bins=80, alpha=0.35, color=intl_colors[strat], density=True, label=strat)
    ax.axvline(x=np.log10(intl_tw_df[strat].median()), color=intl_colors[strat], ls='--', lw=2)

ax.set_xlabel('Terminal Wealth (log\u2081\u2080 scale)', fontsize=12)
ax.set_ylabel('Density', fontsize=12)
ax.set_title('International Monte Carlo: 30-Year Terminal Wealth (pooled from 16 countries)')
ax.legend(fontsize=11)
plt.tight_layout()
plt.show()

print('\n=== International Monte Carlo: 30-Year Terminal Wealth ===')
mc_intl = pd.DataFrame({
    'Mean ($)': intl_tw_df.mean().round(1),
    'Median ($)': intl_tw_df.median().round(1),
    'Mean/Median': (intl_tw_df.mean() / intl_tw_df.median()).round(2),
    'P5 ($)': intl_tw_df.quantile(0.05).round(2),
    'P95 ($)': intl_tw_df.quantile(0.95).round(0),
    'P(Ruin < $1)': (intl_tw_df < 1).mean().round(4),
})
print(mc_intl.to_string())

print('\n--- Ranking Comparison ---')
print('US-only (Part 1) median ranking: 80/20 > 60/40 > Five-Factor > Kelly Barbell > Barbell')
intl_medians = intl_tw_df.median().sort_values(ascending=False)
print(f'International pooled median ranking: {" " > ".join(intl_medians.index.tolist())}')

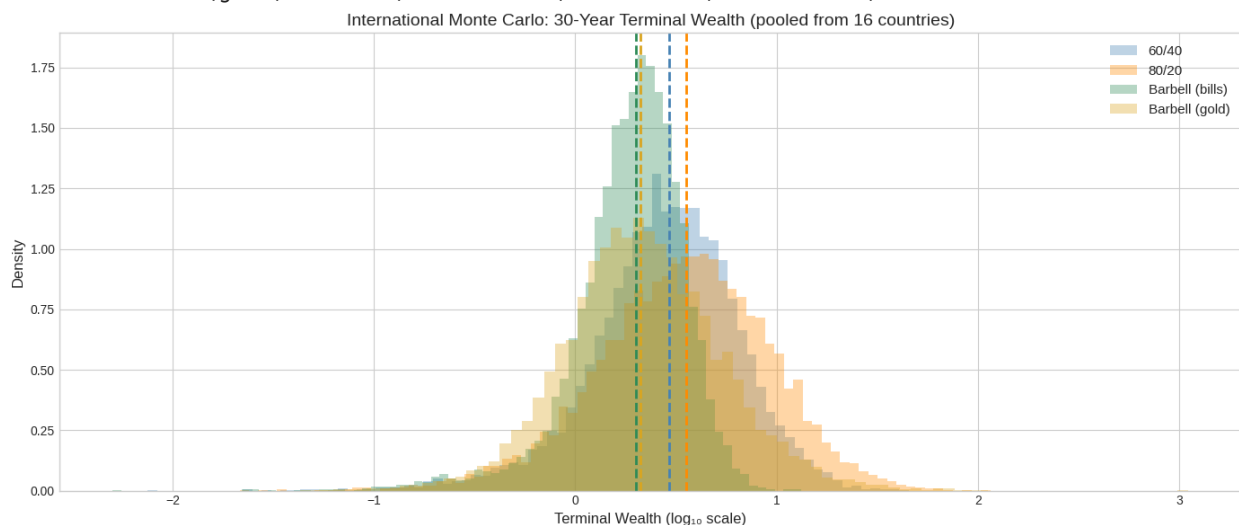
```

=== Pooled Return Summary ===

```

        60/40: n=2203, mean=0.045, std=0.143, min=-0.904, max=0.994
        80/20: n=2203, mean=0.058, std=0.180, min=-0.904, max=1.315
    Barbell (bills): n=2203, mean=0.027, std=0.096, min=-0.904, max=0.757
    Barbell (gold): n=2203, mean=0.042, std=0.246, min=-0.855, max=7.584

```



=== International Monte Carlo: 30-Year Terminal Wealth ===

	Mean (\$)	Median (\$)	Mean/Median	P5 (\$)	P95 (\$)	P(Ruin < \$1)
60/40	3.8	2.9	1.30	0.66	10.0	0.0966
80/20	5.3	3.6	1.50	0.62	16.0	0.1061
Barbell (bills)	2.2	2.0	1.09	0.60	4.0	0.1300
Barbell (gold)	3.5	2.1	1.64	0.51	10.0	0.1803

--- Ranking Comparison ---

US-only (Part 1) median ranking: 80/20 > 60/40 > Five-Factor > Kelly Barbell > Barbell
 International pooled median ranking: 80/20 > 60/40 > Barbell (gold) > Barbell (bills)

Gold Standard vs Floating Era

Gold's behavior differs across monetary regimes, but not in the way you might expect:

Pre-1971 (Gold Standard): Gold's USD price was government-fixed, but its *local-currency price* moved with exchange rates. In countries that maintained their peg to gold/USD, gold's real return was near zero (low inflation, stable exchange rate). In countries that abandoned the peg during crises — Germany, Japan, Italy — gold's local price surged as the exchange rate collapsed. This is gold doing exactly what it should: preserving value when the domestic monetary system fails.

Post-1971 (Floating): After Nixon closed the gold window, gold trades freely in all currencies. Its returns reflect both USD price movements and exchange rate changes. Gold's average real return is higher in this era because it now captures both crisis protection *and* secular demand growth.

The split below shows how strategy performance differs across these two eras.

```
In [21]: # === Pre/Post Gold Standard Split ===

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for ax, (label, year_range) in zip(axes, [('1870-1971 (Gold Standard)', (1870, 1971)),
                                           ('1972-2020 (Floating)', (1972, 2020))]):
    yr_min, yr_max = year_range
    cagr = {s: [] for s in intl_strategies}

    for iso in country_results:
        cr = country_results[iso]
        sub = cr['returns'][cr['returns'].index >= yr_min & (cr['returns'].index <= yr_max)]
        for strat in intl_strategies:
            if strat in sub.columns:
                vals = sub[strat].dropna()
                if len(vals) >= 5:
                    cum = (1 + vals).prod()
                    cagr_val = cum ** (1 / len(vals)) - 1
                    cagr[strat].append(cagr_val)

    means = {s: np.mean(cagr[s]) * 100 if cagr[s] else 0 for s in intl_strategies}
    stds = {s: np.std(cagr[s]) * 100 if len(cagr[s]) > 1 else 0 for s in intl_strategies}

    bars = ax.bar(range(len(intl_strategies)), list(means.values()),
                  color=[intl_colors[s] for s in intl_strategies], alpha=0.8)
    ax.errorbar(range(len(intl_strategies)), list(means.values()),
               yerr=list(stds.values()), fmt='none', color='black', capsize=4)

    ax.set_xticks(range(len(intl_strategies)))
    ax.set_xticklabels(intl_strategies, fontsize=9, rotation=20, ha='right')
    ax.set_ylabel('Mean Real CAGR (%) across countries', fontsize=11)
    ax.set_title(label, fontsize=13)
    ax.axhline(y=0, color='black', ls='--', lw=0.8)
    ax.grid(True, alpha=0.3, axis='y')

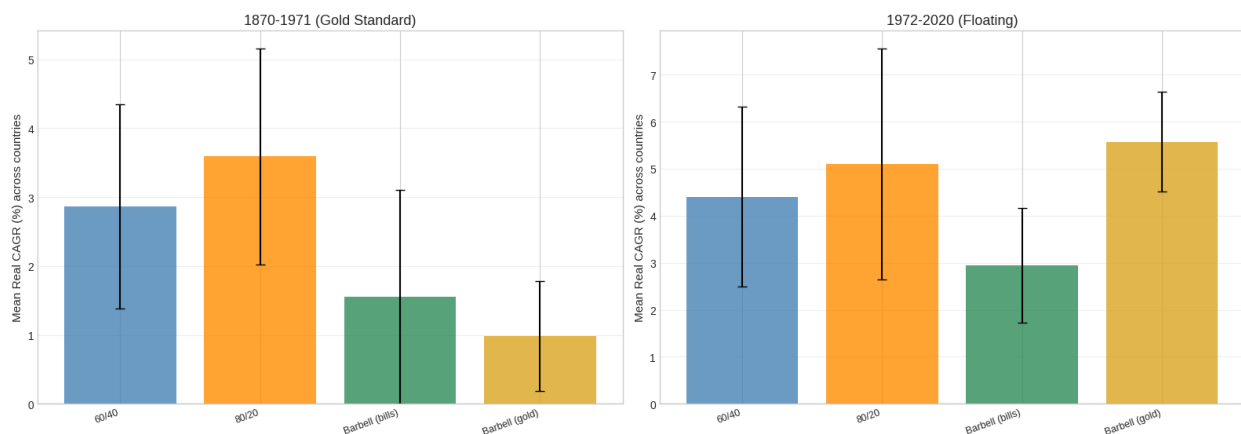
plt.suptitle('Strategy Performance: Gold Standard vs Floating Era', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

```

print('=== Mean Real CAGR by Era (averaged across countries) ===')
for era, (yr_min, yr_max) in [('1870-1971', (1870, 1971)), ('1972-2020', (1972, 2020))]:
    print(f'\n{era}:')
    for strat in intl_strategies:
        cagrs_era = []
        for iso in country_results:
            cr = country_results[iso]
            sub = cr['returns'][(cr['returns'].index >= yr_min) & (cr['returns'].index <= yr_max)]
            if strat in sub.columns:
                vals = sub[strat].dropna()
                if len(vals) >= 5:
                    cum = (1 + vals).prod()
                    cagrs_era.append(cum ** (1 / len(vals)) - 1)
        if cagrs_era:
            print(f'    {strat:>20}: mean={np.mean(cagrs_era):.2%}, '
                  f'std={np.std(cagrs_era):.2%}, n={len(cagrs_era)} countries')

```

Strategy Performance: Gold Standard vs Floating Era



=== Mean Real CAGR by Era (averaged across countries) ===

1870-1971:

60/40: mean=2.87%, std=1.48%, n=16 countries
 80/20: mean=3.59%, std=1.57%, n=16 countries
 Barbell (bills): mean=1.56%, std=1.55%, n=16 countries
 Barbell (gold): mean=0.98%, std=0.80%, n=16 countries

1972-2020:

60/40: mean=4.40%, std=1.92%, n=16 countries
 80/20: mean=5.10%, std=2.45%, n=16 countries
 Barbell (bills): mean=2.94%, std=1.22%, n=16 countries
 Barbell (gold): mean=5.57%, std=1.06%, n=16 countries

Standing on Shoulders

I am not the first to arrive at the conclusion that gold belongs in an all-regime portfolio. Four practitioners built portfolios designed to survive regime uncertainty, and all four include gold:

Harry Browne — Permanent Portfolio (1981): 25% stocks / 25% long-term bonds / 25% gold / 25% cash. Browne framed investing as four economic seasons — prosperity, recession, inflation, deflation — and argued that no one can predict which season comes next. The key insight: *you must hold all four at all times because you cannot predict transitions.*

Ray Dalio — All Weather Portfolio (1996): 30% stocks / 40% long-term bonds / 15% intermediate bonds / 7.5% commodities / 7.5% gold. Dalio's Bridgewater framework uses the same four-quadrant structure (growth up/down × inflation up/down) but sizes allocations by *risk parity* — equalizing the risk contribution of each quadrant rather than the capital. His 2021 book *The Changing World Order* explicitly frames regime transitions as investment-relevant events.

Tyler — Golden Butterfly (PortfolioCharts): 20% stocks / 20% small-cap value / 20% long-term bonds / 20% short-term bonds / 20% gold. A modified Permanent Portfolio with a small-cap value tilt for the prosperity quadrant. Historically delivers ~7% real CAGR with substantially less volatility than 100% equities.

Chris Cole — Dragon Portfolio (Artemis Capital, 2020): 24% stocks / 18% bonds / 21% long volatility / 18% trend following / 19% gold. Cole's "Allegory of the Hawk and Serpent" tests portfolios across 100 years of data including regime transitions. The hawk (secular change) and the serpent (secular growth) require different assets. The dragon navigates both. This is the closest cousin to our gold-barbell: it includes both gold and explicit volatility/convexity exposure.

The common thread: all four allocate 7.5-25% to gold. All four frame the problem as regime uncertainty rather than return optimization. Our JST analysis provides quantitative context for why: in the floating-rate era (post-1971), gold is the safe asset that outperforms bills across 16 countries. The case for gold is conditional on the monetary regime — but since no major economy is returning to the gold standard, the post-1971 evidence is the relevant evidence.

```
In [22]: # === Predecessor Portfolio Comparison ===

predecessor_allocs = {
    'Our Gold-Barbell\n(Part 2)': {'Stocks': 0.15*0.40, 'Bonds': 0, 'Gold': 0.85, 'Cash/Bi'},
    'Permanent Portfolio\n(Browne 1981)': {'Stocks': 0.25, 'Bonds': 0.25, 'Gold': 0.25, 'Ca'},
    'All Weather\n(Dalio 1996)': {'Stocks': 0.30, 'Bonds': 0.55, 'Gold': 0.075, 'Cash/Bills'},
    'Golden Butterfly\n(Tyler)': {'Stocks': 0.40, 'Bonds': 0.40, 'Gold': 0.20, 'Cash/Bills'},
    'Dragon Portfolio\n(Cole 2020)': {'Stocks': 0.24, 'Bonds': 0.18, 'Gold': 0.19, 'Cash/B:
}

categories = ['Stocks', 'Bonds', 'Gold', 'Cash/Bills', 'Other']
cat_colors = {'Stocks': 'steelblue', 'Bonds': 'lightcoral', 'Gold': 'goldenrod',
              'Cash/Bills': 'silver', 'Other': 'mediumpurple'}

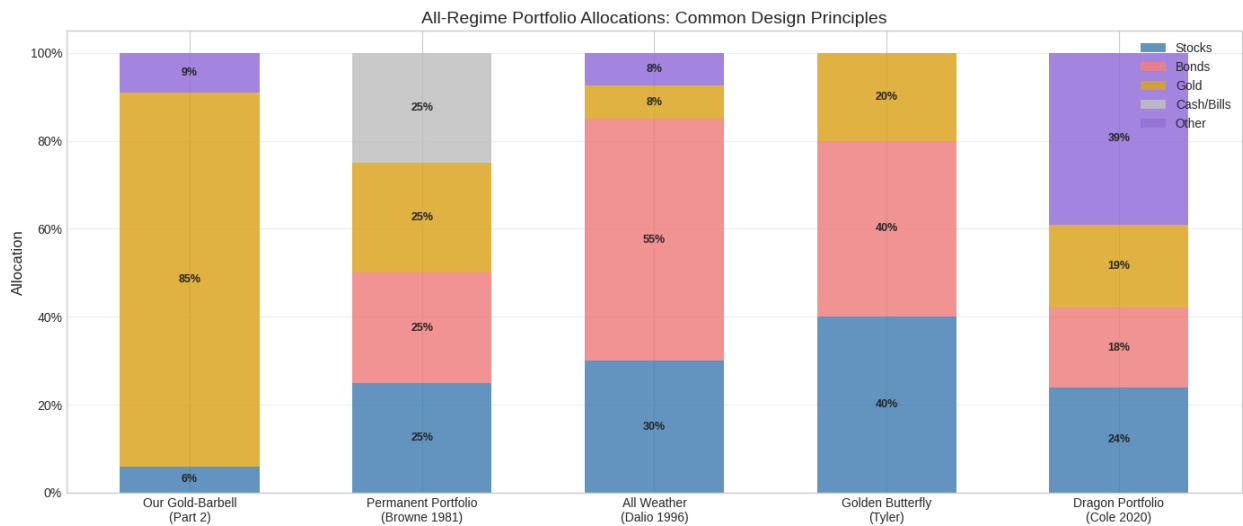
fig, ax = plt.subplots(figsize=(14, 6))
x = np.arange(len(predecessor_allocs))
width = 0.6

bottoms = np.zeros(len(predecessor_allocs))
for cat in categories:
    vals = [predecessor_allocs[name][cat] for name in predecessor_allocs]
    ax.bar(x, vals, width, bottom=bottoms, label=cat, color=cat_colors[cat], alpha=0.85)
    for i, v in enumerate(vals):
        if v >= 0.05:
            ax.text(i, bottoms[i] + v/2, f'{v:.0%}', ha='center', va='center', fontsize=9,
                    bottoms += np.array(vals)

ax.set_xticks(x)
ax.set_xticklabels(list(predecessor_allocs.keys()), fontsize=10)
ax.set_ylabel('Allocation', fontsize=12)
ax.set_title('All-Regime Portfolio Allocations: Common Design Principles', fontsize=14)
ax.legend(loc='upper right', fontsize=10)
ax.set_ylim(0, 1.05)
ax.yaxis.set_major_formatter(PercentFormatter(xmax=1, decimals=0))
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

print('=== Shared Design Principles ===')
print(f'{"Portfolio":>25} {"Gold %":>8} {"Regime Framework":>20} {"Key Insight":>40}')
print('-' * 95)
principles = [
    ('Our Gold-Barbell', '85%', 'Binary (stable/fail)', 'Gold safe asset, leveraged equity'),
    ('Permanent Portfolio', '25%', 'Four seasons', 'Hold all four at all times'),
    ('All Weather', '7.5%', 'Four quadrants', 'Risk parity across regimes'),
    ('Golden Butterfly', '20%', 'Modified seasons', 'SCV tilt for prosperity'),
    ('Dragon Portfolio', '19%', 'Hawk and Serpent', 'Explicit convexity + gold'),
]
```

```
for name, gold_pct, framework, insight in principles:
    print(f'{name:>25} {gold_pct:>8} {framework:>20} {insight:>40}')
```



=== Shared Design Principles ===				
	Portfolio	Gold %	Regime Framework	Key In
sight				

risky	Our Gold-Barbell	85%	Binary (stable/fail)	Gold safe asset, leveraged equity
times	Permanent Portfolio	25%	Four seasons	Hold all four at all
gimes	All Weather	7.5%	Four quadrants	Risk parity across re
erity	Golden Butterfly	20%	Modified seasons	SCV tilt for prosp
gold	Dragon Portfolio	19%	Hawk and Serpent	Explicit convexity +

Conclusions

- No universal optimum exists.** 80/20 is conditionally optimal in countries with stable institutions and sustained equity market growth. The gold-barbell provides protection during regime breaks that equity-heavy and bills-based portfolios cannot. The choice between them is a belief about the likelihood and severity of institutional disruption.
- US-only backtests are survivorship bias.** "Equities always recover" and "80/20 beats everything" are true — in the countries where institutions held. The JST data shows that equities can take decades to recover (Japan 1990-2020), bills can be worthless (Germany 1920s), and the "safe" asset can be the source of ruin. Treating US results as universal is the selection bias that the Investments course warns against in every other context.
- The safe asset is the most important allocation decision.** The literature obsesses over the equity-bond split. The Part 2 analysis shows that the choice of *safe asset* — bills vs gold — determines whether the portfolio survives the scenario it was designed to protect against. A bills-barbell fails in exactly the scenario (regime collapse, currency debasement) where a barbell is supposed to provide protection. A gold-barbell survives because gold's local-currency price rises when the exchange rate collapses.
- Every serious all-regime portfolio includes gold.** Browne, Dalio, Tyler, and Cole arrived at this conclusion independently through different analytical frameworks. Our JST analysis across 16 countries and 150 years shows why: gold is the only widely available asset whose value is not tied to a single government's promises. When computed in local-currency terms via exchange rates,

gold preserved purchasing power through the worst regime transitions in modern history.

Limitations

- **Annual frequency** in the JST data misses within-year drawdowns and volatility, likely understating the true pain of each strategy.
- **Gold pre-1971** is approximated as –inflation (fixed nominal price). This is conservative — gold standard abandonments typically produced gold appreciation in local currency terms, which would improve the gold-barbell's performance during crises.
- **Leverage may not have been historically implementable** in many JST countries. The 2x leveraged equity assumption is a modeling choice, not a historical claim.
- **Risky sleeve simplified** to 2x equity for international data (no treasuries, insurance, or multi-asset convexity). This understates the full barbell's convexity relative to Part 1.
- **Transaction costs ignored** throughout. Real implementation of any leveraged strategy involves borrowing costs, bid-ask spreads, and tax drag.
- **JST ends in 2020**. Post-2020 inflation, the Ukraine war, and central bank tightening are not captured.
- **Bootstrap Monte Carlo** preserves the marginal distribution but destroys temporal dependence. Real-world drawdowns may be deeper and longer.
- **Part 1 results** (Five-Factor, Kelly Barbell) depend on the specific construction methodology. The Five-Factor's rolling optimization introduces estimation noise; the Kelly Barbell's trailing window and cap are parameters that could be tuned differently.
- The 60/40 and 80/20 remain the hardest benchmarks to beat — they are deterministic, cheap, and have no parameters to overfit.

References

- Jordà, Ò., Schularick, M. & Taylor, A.M. (2017). "Macrofinancial History and the New Business Cycle Facts." *NBER Macroeconomics Annual*, 31(1), 213-263.
- Dimson, E., Marsh, P. & Staunton, M. (2002). *Triumph of the Optimists: 101 Years of Global Investment Returns*. Princeton University Press.
- Browne, H. (1981). *Inflation-Proofing Your Investments*. (Permanent Portfolio concept.)
- Dalio, R. / Bridgewater Associates (1996). All Weather strategy. See also: Dalio, R. (2021). *The Changing World Order*.
- Cole, C. / Artemis Capital Management (2020). "The Allegory of the Hawk and Serpent: How to Grow and Protect Wealth for 100 Years."
- Erb, C.B. & Harvey, C.R. (2013). "The Golden Dilemma." *Financial Analysts Journal*, 69(4), 10-42.
- Peters, O. (2019). "The Ergodicity Problem in Economics." *Nature Physics*, 15, 1216-1221.
- Taleb, N.N. (2012). *Antifragile: Things That Gain from Disorder*. Random House.
- Spitznagel, M. (2021). *Safe Haven: Investing for Financial Storms*. Wiley.
- Hannigan, K. et al. (2026). "Buy the Tulip: CAPM Under Fat Tails." Investments, Tuck School of Business.
- Kelly, J.L. (1956). "A New Interpretation of Information Rate." *Bell System Technical Journal*, 35, 917-926.
- Fama, E.F. & French, K.R. (1993, 2015). Factor model papers.
- Rockafellar, R.T. & Uryasev, S. (2000). "Optimization of Conditional Value-at-Risk." *Journal of Risk*, 2, 21-41.